🦉 학생용 따라하기 워크북

전국 급식 비교 앱 만들기 — 코딩을 하나도 몰라도 괜찮아요

이름 _____ **반** _____ **번호** _____

이 워크북은 이렇게 되어 있어요

25개의 작은 계단이 있습니다. 한 계단마다 이렇게 합니다.

```
① 코드를 복사해서 붙여넣는다
      ↓
② 저장한다 (Ctrl + S)
      ↓
③ 새로고침한다 (F5)
      ↓
④ 👀 화면에서 결과를 눈으로 확인한다   ← 여기가 제일 중요!
      ↓
⑤ □ 에 체크하고 다음 계단으로
```

한 계단은 **3~7분**이면 끝납니다. 매번 화면이 바뀌는 게 보이니까 지루하지 않아요.

> 🆘 **가장 먼저 알아둘 것 — 막혀도 괜찮아요!**
>
> `steps` 폴더에 **각 장의 정답 파일**이 들어 있습니다. 도저히 안 되면 그 파일을 통째로 복사해서 내 `app.js`에 붙여넣으면 **바로 다음 장을 같이 할 수 있습니다.** 이건 부끄러운 게 아니에요. **끝까지 같이 가는 게 훨씬 중요합니다!**

📋 진도표 — 할 때마다 체크하세요

0장 준비운동

- ☐ 준비 1. 앱 화면 구경하기
- ☐ 준비 2. app.js 열기
- ☐ 준비 3. 8개의 칸 만들기

1장 화면 깨우기 ← 여기부터 진짜 코딩!

- ☐ 미션 1. 화면에 인사말 띄우기
- ☐ 미션 2. 오늘 날짜 자동으로 넣기
- ☐ 미션 3. 전국 17개 교육청 목록 만들기
- ☐ 미션 4. 버튼 눌리게 하기

2장 학교 찾아오기

- ☐ 미션 5. 인터넷에서 진짜 자료 구경하기

- [ ] 미션 6. 학교를 검색해서 F12 창에 띄우기
- [ ] 미션 7. 학교를 눈으로 보고 고르기
- [ ] 미션 8. 인증키 넣기

3장 급식 불러오기

- [ ] 미션 9. 우리 학교 급식을 F12 창에 띄우기
- [ ] 미션 10. 긴 글자를 쪼개기
- [ ] 미션 11. 자료를 정리해서 상자에 담기
- [ ] 미션 12. **화면에 식단 띄우기** ⭐
- [ ] 미션 13. 알레르기 번호 붙이기
- [ ] 미션 14. 원산지 표 만들기

4장 그래프 그리기

- [ ] 미션 15. 영양소 막대그래프 그리기 ⭐
- [ ] 미션 16. 요리를 식품군으로 자동 분류하기
- [ ] 미션 17. 식품군 색깔 띠 만들기 + 🔍 컴퓨터의 실수 찾기

5장 점수 만들기

- [ ] 미션 18. 급식 점수 매기기 ⭐
- [ ] 미션 19. 거미줄 차트 그리기
- [ ] 미션 20. 내 기준으로 순위 뒤집기 ⭐
- [ ] 미션 21. 자동 분석 글 만들기
- [ ] 미션 22. 마무리 기능 5가지

📖 0장. 준비운동

먼저 알아야 할 낱말 (딱 12개만!)

지금 다 외우지 마세요. **모르는 말이 나오면 여기로 돌아와서 찾아보면 됩니다.**

낱말	아주 쉬운 뜻	예
코드	컴퓨터에게 시키는 말	`$('#date').value = '2026-07-08';`
파일	코드를 적어 두는 종이	`app.js`
함수	이름을 붙여 둔 **기계**	`todayISO()` 를 부르면 오늘 날짜가 나옴
부른다(호출)	그 기계를 작동시킨다	`todayISO()` 처럼 뒤에 `()` 를 붙임
변수	값을 담아 두는 **상자**	`const 나이 = 15;`
문자열	글자 덩어리. 따옴표로 감쌈	`'안녕'`
배열	여러 개를 순서대로 담은 줄	`['사과', '우유', '밥']`
객체	이름표를 붙여 담은 상자	`{ 이름: '홍길동', 나이: 15 }`
세미콜론 `;`	한 문장의 마침표	줄 끝에 붙임
중괄호 `{ }`	여러 줄을 하나로 묶는 보자기	`function 이름() { 여기 }`
주석	컴퓨터가 **무시하는** 메모	`// 이건 설명이에요`
콘솔(Console)	컴퓨터의 **속마음 창**. F12로 엶	결과를 몰래 확인할 때

🔰 따옴표가 3가지예요!

기호	이름	언제 쓰나
`'안녕'`	작은따옴표	보통 글자
`"안녕"`	큰따옴표	보통 글자 (작은따옴표와 같음)
`` `안녕 ${이름}` ``	**백틱**	글자 **사이에 값을 끼워 넣을 때**

백틱은 키보드 왼쪽 위, 숫자 1 왼쪽에 있는 `~` 키를 그냥 누르면 나옵니다. 백틱 안에서 `${ }` 를 쓰면 그 자리에 값이 들어갑니다.

```
const 이름 = '민서';
console.log(`안녕 ${이름}!`);   // → 안녕 민서!
```

꼭 외워야 할 단축키 4개

단축키	하는 일	언제
Ctrl + S	저장	코드를 고친 **직후 항상**
F5	새로고침	저장한 다음 **항상**
F12	개발자 도구 열기	결과를 확인하거나 오류를 볼 때
Ctrl + A → Ctrl + V	전부 선택 후 붙여넣기	정답 파일로 갈아탈 때

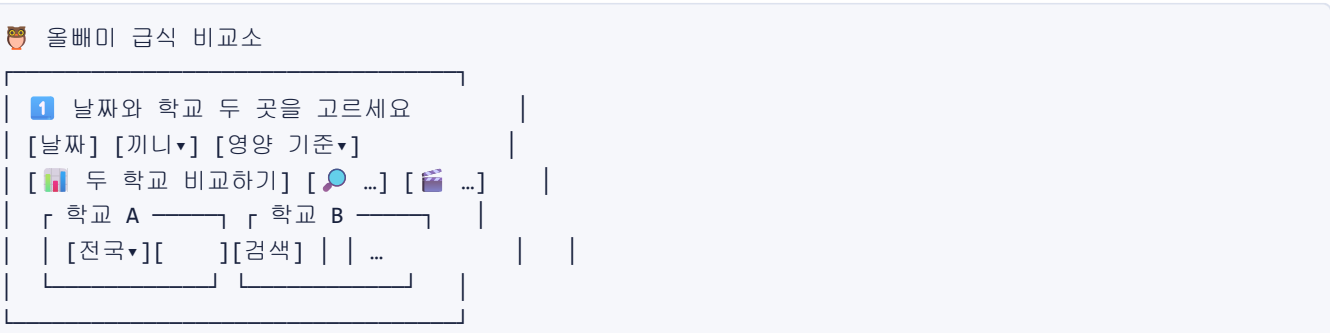
> ⚠️ **가장 많이 하는 실수 1위**
>
> **저장(Ctrl+S)을 안 하고 새로고침(F5)하기!** 저장을 안 하면 아무리 새로고침해도 화면이 안 바뀝니다. **고쳤으면 → Ctrl+S → F5.** 이 순서를 입으로 말하면서 하세요.

✅ 준비 1. 앱 화면 구경하기 (코드 0줄!)

할 일

1. 바탕화면의 `급식비교` 폴더를 엽니다.
2. `index.html` 을 **두 번 클릭**합니다.

👀 **확인!** 인터넷 창이 열리고 이런 화면이 보이면 성공입니다.



☐ **성공!** 화면이 보였어요.

> 🔰 **지금 상태**: 화면(집)은 다 지어져 있는데, **전기가 안 들어온 상태**예요. 버튼을 눌러도 아무 일도 일어나지 않습니다. 우리가 할 일이 바로 **전기를 넣는 것**입니다. 화면을 만드는 `index.html`(뼈대)과 `style.css`(꾸미기)는 이미 완성되어 있으니 **건드리지 않습니다.** 우리가 만드는 건 `app.js` **딱 하나**예요!

✅ 준비 2. app.js 열기

할 일

1. `급식비교` 폴더에서 `app.js` 를 찾습니다.

2. **마우스 오른쪽 클릭 → 연결 프로그램 → Visual Studio Code** (VS Code가 없으면 **메모장**도 괜찮아요)
3. 파일 안에 글자가 잔뜩 있으면 **Ctrl + A** (전체 선택) → **Delete** 로 **전부 지웁니다.**

👀 **확인!**

☐ app.js가 열렸고, 안이 **텅 비어** 있어요.

> 🔰 **왜 지우나요?** 이 파일에는 이미 완성된 정답이 들어 있습니다. 우리는 **처음부터 직접 만들어 볼 거예요.** 지워도 걱정 마세요. 정답은 `steps` 폴더에 그대로 남아 있습니다!

✅ 준비 3. 8개의 칸 만들기 (아주 중요!)

앞으로 코드를 계속 붙여넣을 건데, **어디에 넣을지** 헷갈리면 안 되겠죠? 그래서 **미리 8개의 칸을 만들어 둡니다.** 앞으로는 "③번 칸에 붙여넣으세요"라고만 하면 돼요.

할 일 — 아래를 **전부 복사해서** 빈 `app.js`에 붙여넣으세요.

```
'use strict';

/* ===== ① 도구 칸 ===== */


/* ===== ② 자료 칸 ===== */


/* ===== ③ NEIS 부르기 칸 ===== */


/* ===== ④ 글자 쪼개기 칸 ===== */


/* ===== ⑤ 계산하기 칸 ===== */


/* ===== ⑥ 화면 그리기 칸 ===== */


/* ===== ⑦ 버튼 연결 칸 ===== */


/* ===== ⑧ 시작 칸 ===== */
function init() {

  /* ★★★ 앞으로 여기에 계속 한 줄씩 추가합니다 ★★★ */

}

init();
```

💾 **저장(Ctrl+S)** → 🔄 **새로고침(F5)**

👀 **확인!**

1. **F12** 를 누릅니다.
2. 위쪽 탭에서 **Console** 을 클릭합니다.
3. **빨간 글씨(오류)가 없으면 성공**입니다. (아무것도 안 나와도 정상이에요!)

▢ **성공!** 빨간 글씨가 없어요.

> 🔰 **방금 붙여넣은 것 설명**
>
> | 코드 | 뜻 |
> | --- | --- |
> | `'use strict';` | "실수를 엄격하게 잡아 줘" 라는 뜻. 맨 위에 한 번만 씁니다 |
> | `/* ===== ① 도구 칸 ===== */` | **주석**(메모)입니다. 컴퓨터는 그냥 무시해요. 우리 눈에만 보입니다 |
> | `function init() { }` | `init`(이닛)이라는 이름의 **기계**를 만든 것. 아직 속은 비어 있어요 |
> | `init();` | 그 기계를 **작동시키는** 한 줄. **이 줄이 없으면 아무 일도 안 일어납니다!** |
>
> `init` 은 **시작(initialize)** 이라는 뜻이에요. 앱이 켜질 때 제일 먼저 하는 일들을 여기에 적습니다.

🚀 1장. 화면 깨우기

이제 진짜 코딩입니다! 이 장이 끝나면 화면이 **살아 움직입니다.**

✅ 미션 1. 화면에 인사말 띄우기

무엇을 하나요? 화면에 내가 쓴 글씨가 나타나게 합니다.

✍️ 1) ①번 도구 칸에 붙여넣기

```
/* 화면에서 무언가를 찾아 주는 도구 (앞으로 계속 씁니다!) */
const $  = (sel, root = document) => root.querySelector(sel);
const $$ = (sel, root = document) => Array.from(root.querySelectorAll(sel));

/* 화면 위쪽 알림줄에 글씨를 띄우는 기계 */
function setStatus(html, kind = '') {
  const box = $('#status');
  if (!html) { box.hidden = true; box.innerHTML = ''; return; }
  box.hidden = false;
  box.className = 'status ' + kind;
  box.innerHTML = html;
}
```

✍️ 2) ⑧번 시작 칸의 `★★★` 표시 아래에 붙여넣기

```
  setStatus('안녕하세요! 제가 만드는 급식 앱입니다. 🦉');
```

💾 **Ctrl+S** → 🔄 **F5**

👀 **확인!** 화면 가운데쯤에 **"안녕하세요! 제가 만드는 급식 앱입니다. 🦉"** 라는 상자가 나타납니다.

▢ **성공!** 내가 쓴 글씨가 화면에 떴어요! 🎉

🎨 **재미로 해보기** — 따옴표 안의 글씨를 **내 이름**으로 바꿔 보세요. `setStatus('민서가 만드는 급식 앱! 🦉');` → 저장 → F5

> 🔰 **코드 뜯어보기**
>
> ① `$` **는 뭐예요?** 화면에서 물건을 찾아오는 **집게**라고 생각하세요. 원래 이름은 `document.querySelector` 인데 너무 길어서 `$` 로 줄였습니다.
>
> ```
> $('#status')  →  "id가 status 인 것을 찾아와"
> ```
>
> - `#` 로 시작하면 → **id**를 찾음 (id는 반에 한 명뿐인 이름표)
> - `.` 로 시작하면 → **class**를 찾음 (class는 여러 명이 같이 다는 이름표)
>
> `index.html` 파일 안에 `<p class="status" id="status">` 라고 적혀 있어서 찾을 수 있는 거예요.
>
> ② `function setStatus(html, kind = '') {` **는요?** `setStatus` 라는 **기계를 만든다**는 뜻입니다. 괄호 안의 `html`, `kind` 는 **기계에 넣는 재료**예요. `kind = ''` 는 "재료를 안 넣으면 빈 글자로 하겠다"는 뜻입니다.
>
> ③ `box.innerHTML = html;` `innerHTML` 은 **"그 상자 안의 내용"** 이라는 뜻이에요. 여기에 글자를 넣으면 화면에 그대로 보입니다.
>
> ④ `if (!html) { ... return; }` `if` 는 **만약에**, `!` 는 **아니라면** 이라는 뜻이에요. "만약 글자가 없으면 → 상자를 숨기고 → `return` (여기서 끝!)"

✅ 미션 2. 오늘 날짜 자동으로 넣기

무엇을 하나요? 날짜 칸에 오늘 날짜가 저절로 들어가게 합니다.

✍️ 1) ①번 도구 칸 맨 아래에 붙여넣기

```
/* 오늘 날짜를 2026-07-08 모양으로 만들어 주는 기계 */
function todayISO() {
  const d = new Date();
  const p = n => String(n).padStart(2, '0');
  return `${d.getFullYear()}-${p(d.getMonth() + 1)}-${p(d.getDate())}`;
}
```

✍️ 2) ⑧번 시작 칸의 `★★★` 아래에 한 줄 추가

```
  $('#date').value = todayISO();
```

💾 **Ctrl+S** → 🔄 **F5**

👀 **확인!** 화면 왼쪽 위 **날짜 칸에 오늘 날짜**가 들어가 있습니다.

▢ **성공!** 날짜가 자동으로 채워졌어요.

> 🔰 **코드 뜯어보기**
>
> `new Date()` → 지금 이 순간의 날짜와 시각을 가져옵니다.
>
> `d.getMonth() + 1` ← 여기 주의! 컴퓨터는 달(月)을 **0부터** 셉니다. 1월이 0, 8월이 7이에요. 그래서 **+1**을 해 줘야 우리가 아는 달이 됩니다. (많은 사람이 틀리는 부분!)
>
> `padStart(2, '0')` → 글자를 2칸으로 만들고 빈 곳은 `0`으로 채웁니다. `8` → `08` 이 됩니다. (날짜는 `2026-7-8`이 아니라 `2026-07-08` 이어야 하니까요)
>
> `.value` → 입력칸에 **쓰여 있는 값**을 뜻합니다. 여기에 값을 넣으면 칸이 채워져요.
>
> `return` → 기계가 만들어 낸 결과를 **밖으로 내보내는** 것입니다.

✅ 미션 3. 전국 17개 교육청 목록 만들기

무엇을 하나요? 학교 A와 B의 지역 선택 목록을 채웁니다.

✍️ 1) ②번 자료 칸에 붙여넣기

```
/* 17개 시도교육청 코드. NEIS가 정해 둔 약속이라 그대로 적어야 합니다. */
const SIDO_LIST = [
  ['', '전국(시도 상관없이)'],
  ['B10', '서울특별시교육청'], ['C10', '부산광역시교육청'],
  ['D10', '대구광역시교육청'], ['E10', '인천광역시교육청'],
  ['F10', '광주광역시교육청'], ['G10', '대전광역시교육청'],
  ['H10', '울산광역시교육청'], ['I10', '세종특별자치시교육청'],
  ['J10', '경기도교육청'],     ['K10', '강원특별자치도교육청'],
  ['M10', '충청북도교육청'],   ['N10', '충청남도교육청'],
  ['P10', '전북특별자치도교육청'], ['Q10', '전라남도교육청'],
  ['R10', '경상북도교육청'],   ['S10', '경상남도교육청'],
  ['T10', '제주특별자치도교육청'],
];
```

✍️ 2) ⑧번 시작 칸의 `★★★` 아래에 추가

```
  // 🧪 실험용 — 미션 7에서 지웁니다
  $$('.sido').forEach(sel => {
    sel.innerHTML = SIDO_LIST
      .map(([code, name]) => `<option value="${code}">${name.replace('교육청', '')}</option>`)
      .join('');
  });
```

💾 **Ctrl+S** → 🔄 **F5**

👀 **확인!** 학교 A 상자의 `전국(시도 상관없이)` **목록을 클릭**해 보세요. 서울특별시부터 제주특별자치도까지 **18개**가 쭉 나옵니다. (전국 1개 + 교육청 17개)

▢ **성공!** 목록이 18개 나와요.

> 🔰 **코드 뜯어보기**
>
> `const SIDO_LIST = [ ... ];` `const` 는 **바뀌지 않는 상자**를 만든다는 뜻. `[ ]` 는 **배열**(줄 세우기)입니다. 안에 또 `[ ]` 가 있죠? `['B10', '서울특별시교육청']` 처럼 **짝**으로 묶어 둔 거예요.
>
> `$$('.sido')` `$$` 는 **여러 개**를 찾는 집게였죠. `.sido` 는 class가 sido인 것 → 학교 A와 B의 목록 **2개**를 찾습니다.
>
> `.forEach(sel => { ... })` "찾은 것 **하나하나마다** 이걸 해라" 라는 뜻이에요. 2개니까 2번 실행됩니다.
>
> `.map()` 과 `.join('')` — **앞으로 계속 나오니 여기서 확실히!**
>
> ```
> [['B10','서울특별시교육청'], ['C10','부산광역시교육청']]
>           ↓ .map()  = 하나하나를 다른 모양으로 바꾸기
> ['<option value="B10">서울특별시</option>', '<option value="C10">부산광역시</option>']
>           ↓ .join('')  = 전부 이어 붙여 하나의 긴 글자로
> '<option value="B10">서울특별시</option><option value="C10">부산광역시</option>'
> ```
>
> 이렇게 만든 긴 글자를 `innerHTML` 에 넣으면 → 화면에 목록이 생깁니다!
>
> `name.replace('교육청', '')` → `'서울특별시교육청'` 에서 `'교육청'` 을 지워 `'서울특별시'` 로 만듭니다.

✅ 미션 4. 버튼 눌리게 하기

무엇을 하나요? 버튼을 누르면 화면이 반응하게 합니다.

✍️ ⑧번 시작 칸의 `★★★` 아래에 추가

```
// 🧪 실험용 — 미션 9에서 바꿉니다
$('#compareBtn').addEventListener('click', () => {
  setStatus(`버튼이 눌렸습니다! 고른 날짜는 <b>${$('#date').value}</b> 입니다. 🎉`);
});
```

💾 **Ctrl+S** → 🔄 **F5**

👀 **확인!** 📊 **두 학교 비교하기** 버튼을 눌러 보세요. 알림줄 글씨가 **"버튼이 눌렸습니다! 고른 날짜는 2026-... 입니다. 🎉"** 로 바뀝니다. 날짜 칸을 바꾸고 다시 눌러 보세요. 글씨의 날짜도 같이 바뀝니다!

▢ **성공!** 버튼이 반응해요.

> 🔰 **코드 뜯어보기**
>
> `addEventListener('click', ...)` "**클릭**이라는 일이 생기면 이걸 해라" 라고 **귀를 달아 두는 것**입니다. (event = 사건, listener = 듣는 사람 → **사건 감시원**)
>
> `() => { ... }` 이름 없는 **일회용 기계**입니다. 버튼이 눌릴 때만 작동해요. (화살표 함수라고 불러요)
>
> `<b>` **는 뭐죠? 굵은 글씨**로 만드는 HTML 표시입니다. `setStatus` 는 글자를 화면에 그대로 그려 주니까 `<b>...</b>` 로 감싸면 진하게 나옵니다.

🎊 1장 완성! 여기까지 왔으면 정말 잘한 거예요

지금 되는 것
✅ 화면에 내 글씨 띄우기
✅ 오늘 날짜 자동 입력
✅ 전국 교육청 목록
✅ 버튼 반응

🆘 **못 따라왔다면** → `steps/1단계_app.js` 를 열어 **Ctrl+A** → **Ctrl+C**, 내 `app.js`에서 **Ctrl+A** → **Ctrl+V**. 저장하고 다음 장으로 가세요! (내가 만든 것과 조금 다를 수 있지만, 다음 장을 하는 데는 전혀 문제없어요)

🌐 2장. 학교 찾아오기

이제 **인터넷에서 진짜 자료**를 가져옵니다. 이 수업의 핵심이에요!

✅ 미션 5. 인터넷에서 진짜 자료 구경하기 (코드 0줄!)

코드를 쓰기 전에, **우리가 가져올 자료가 어떻게 생겼는지** 먼저 눈으로 봅시다.

할 일 인터넷 주소창에 아래를 **그대로 복사해서 붙여넣고 엔터**를 누르세요.

```
https://open.neis.go.kr/hub/schoolInfo?Type=json&pSize=5&SCHUL_NM=창원고등학교
```

👀 **확인!** 글자가 잔뜩 나옵니다. 이렇게요.

```
{"schoolInfo":[{"head":[{"list_total_count":1}, … ]},
{"row":[{"ATPT_OFCDC_SC_CODE":"S10","SCHUL_NM":"창원고등학교", … }]}]}
```

▢ **성공!** 글자 덩어리가 보여요.

🎯 **꼭 해보기** — 주소 맨 뒤 `창원고등학교` 를 **우리 학교 이름**으로 바꿔서 다시 해보세요!

🔰 방금 무슨 일이 있었나요?

주소 하나로 전국 학교 정보를 받아왔습니다. 이것이 바로 **API**예요. API = 프로그램끼리 자료를 주고받는 **약속**

주소를 뜯어보면 이렇습니다.

```
https://open.neis.go.kr/hub/schoolInfo ? Type=json & pSize=5 & SCHUL_NM=창원고등학교
└──────────── 어디에 물어볼까 ────────────┘ ↑ └조건1┘ ↑ └조건2┘ ↑ └──── 조건3 ────┘
                                          물음표로 시작    &로 이어 붙임
```

조건	뜻
`Type=json`	JSON이라는 모양으로 줘
`pSize=5`	5줄만 줘
`SCHUL_NM=창원고등학교`	이 이름의 학교를 찾아 줘

JSON(제이슨)은 자료를 적는 **정해진 모양**이에요. `{ }` 는 이름표가 붙은 상자, `[ ]` 는 줄 세운 목록. 우리가 배운 객체와 배열이죠!

✅ 미션 6. 학교를 검색해서 F12 창에 띄우기

무엇을 하나요? 방금 주소창으로 한 일을 **코드로** 합니다.

✍️ 1) ①번 도구 칸 맨 아래에 붙여넣기

```
/* 인터넷에서 온 글자를 화면에 안전하게 넣기 위한 기계 */
function esc(s) {
  return String(s ?? '').replace(/[&<>"']/g, c =>
    ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
}
```

✍️ 2) ③번 NEIS 부르기 칸에 붙여넣기

```
const API_BASE = 'https://open.neis.go.kr/hub/';

/* 저장해 둔 인증키 꺼내기 (없으면 빈 글자) */
function getKey() {
  return localStorage.getItem('neisKey') || '';
}

/* NEIS에 물어보고 답을 받아오는 기계 */
async function callNeis(endpoint, params) {
  const q = new URLSearchParams({ Type: 'json', pIndex: 1, pSize: 100, ...params });
  const key = getKey();
  if (key) q.set('KEY', key);

  const url = API_BASE + endpoint + '?' + q.toString();
  console.log('부르는 주소 👉', url);

  let res;
  try {
    res = await fetch(url);                 // ← 인터넷에서 받아오기!
  } catch (e) {
    throw new Error('NETWORK');
  }
  if (!res.ok) throw new Error('HTTP ' + res.status);

  const data = await res.json();            // ← 글자를 자료로 바꾸기

  // 자료가 하나도 없으면 NEIS는 { RESULT: ... } 만 보냅니다
  if (data.RESULT) {
    if (data.RESULT.CODE === 'INFO-200') return [];
    throw new Error(data.RESULT.CODE + ' : ' + data.RESULT.MESSAGE);
  }

  const body = data[endpoint];
  if (!body) return [];

  const head = body[0]?.head?.[1]?.RESULT;
  if (head && head.CODE !== 'INFO-000') throw new Error(head.CODE + ' : ' + head.MESSAGE);

  const rows = body[1]?.row || [];
  rows.totalCount = body[0]?.head?.[0]?.list_total_count ?? rows.length;
  return rows;
}

/* 학교 이름으로 찾기 */
function searchSchools(name, sido) {
  const params = { SCHUL_NM: name };
  if (sido) params.ATPT_OFCDC_SC_CODE = sido;
  return callNeis('schoolInfo', params);
}

/* 문제가 생겼을 때 알아들을 수 있는 말로 바꿔 주기 */
function showError(e) {
  const msg = String(e.message || e);
```

```
  if (msg === 'NETWORK') {
    setStatus(`<b>NEIS에 연결하지 못했습니다.</b>
      <ol>
        <li>인터넷이 연결되어 있는지 확인하세요.</li>
        <li>학교 방화벽이 막을 수 있습니다. 휴대폰 핫스팟으로 시도해 보세요.</li>
        <li>그래도 안 되면 <b>🎬 예시 데이터로 먼저 보기</b> 버튼을 누르면 수업을 이어갈 수 있습니다.
</li>
      </ol>`, 'error');
    return;
  }
  if (msg.includes('INFO-300')) {
    setStatus('<b>인증키가 올바르지 않습니다.</b> 🔑 상자를 열어 키를 다시 붙여넣거나, 키를 지우고(키 없
이) 사용하세요.', 'error');
    return;
  }
  if (msg.includes('ERROR-337')) {
    setStatus('<b>오늘 사용할 수 있는 횟수를 다 썼습니다.</b> 인증키를 등록하면 훨씬 많이 쓸 수 있습니
다.', 'error');
    return;
  }
  setStatus(`문제가 생겼습니다: <code>${esc(msg)}</code>`, 'error');
}
```

✍️ 3) ⑧번 시작 칸의 ★★★ 아래에 추가

```
  // 💉 실험용 — 미션 7에서 지웁니다
  $$('.searchBtn').forEach(btn => {
    btn.addEventListener('click', async () => {
      const box = btn.closest('.picker');          // 이 버튼이 들어 있는 상자
      const q   = $('.q', box).value;              // 그 상자의 입력칸 글자
      setStatus(`‘${esc(q)}’ 검색 중…`, 'loading');
      try {
        const rows = await searchSchools(q, $('.sido', box).value);
        console.log('찾은 학교 👉', rows);
        setStatus(`${rows.length}곳을 찾았습니다! <b>F12 → Console</b> 을 열어 보세요. 🔍`);
      } catch (e) { showError(e); }
    });
  });
```

💾 Ctrl+S → 🔄 F5

👀 **확인!** (두 가지를 봅니다)

1. 학교 A 입력칸에 `창원고등학교` 를 넣고 **검색** 버튼을 누릅니다.
2. 알림줄이 **파란색**으로 "검색 중…" → 잠시 후 **"1곳을 찾았습니다!"** 로 바뀝니다.
3. **F12** → **Console** 탭을 봅니다.

```
부르는 주소 👉 https://open.neis.go.kr/hub/schoolInfo?Type=json&…
찾은 학교 👉 ▶ Array(1)
```

4. `▶ Array(1)` 의 **▶ 삼각형을 클릭**해서 펼쳐 보세요. `SCHUL_NM: "창원고등학교"`, `SD_SCHUL_CODE: "9010114"` 같은 게 보입니다!

▢ **성공!** 인터넷에서 진짜 학교 정보를 가져왔어요! 🎉 🎉

> 🔰 **코드 뜯어보기 — 여기가 오늘의 산!**
>
> ① `async` **와** `await` **— 분식집 이야기** 🍜
>
> ```
> await 없으면:  라면 주문! → 곧바로 "맛있다!" 라고 말함 → 아직 라면이 안 나왔는데?? 😵
> await 있으면:  라면 주문! → 나올 때까지 기다림 → 받고 나서 "맛있다!" 😊
> ```
>
> 인터넷은 느립니다(0.5초쯤). `await` 를 안 쓰면 자료가 오기 전에 다음 줄로 넘어가 버려요. **`await` 를 쓰는 기계 앞에는 반드시 `async` 를 붙여야 합니다.**
>
> ② `fetch(url)` 진짜로 인터넷에서 받아오는 명령입니다. 딱 한 줄이에요!
>
> ③ `await res.json()` 받아온 **글자**를 우리가 쓸 수 있는 **자료(객체·배열)** 로 바꿉니다.
>
> ④ **껍데기가 두 겹인 이유** NEIS의 답은 이렇게 생겼어요.
>
> ```
> { "schoolInfo": [ { "head": [...] },     ← 0번: 몇 개 찾았는지 알려 주는 칸
>                   { "row":  [...] } ] }  ← 1번: 진짜 자료가 든 칸
> ```
>
> 그래서 `body[1].row` 로 들어가야 학교 목록이 나옵니다. **0번부터 세는 것**을 잊지 마세요!
>
> ⑤ `?.` **는 안전벨트** `body[1]?.row` → "만약 `body[1]` 이 없어도 오류 내지 말고 그냥 넘어가"
>
> ⑥ `try` / `catch`
>
> - `try` → 일단 해 봐
> - `catch` → 잘못되면 여기로 와 (**프로그램이 죽지 않게!**)
>
> ⑦ `btn.closest('.picker')` "내가 눌린 버튼을 **감싸고 있는** 상자를 찾아라". 학교 A 버튼을 누르면 A 상자를 찾습니다.
>
> ⑧ `console.log('찾은 학교 👉', rows)` **콘솔(속마음 창)에 몰래 찍어 보는 것**입니다. 화면에는 안 보이고 F12에서만 보여요. 코딩할 때 **가장 많이 쓰는 확인 방법**이니 꼭 기억하세요!

😵 **"5곳만 나와요"** → 오류가 아닙니다! 인증키가 없으면 NEIS가 5줄만 줍니다. 미션 8에서 해결해요.

✅ 미션 7. 학교를 눈으로 보고 고르기

무엇을 하나요? 검색 결과를 **화면 목록**으로 보여 주고, 클릭하면 선택되게 합니다.

✍️ 1) 먼저 ⑧번 시작 칸에서 🧪 실험용 코드 2개를 지웁니다

지울 것 ① (미션 3에서 넣은 것)

```
// 🧪 실험용 — 미션 7에서 지웁니다
$$('.sido').forEach(sel => { … });     ← 이 덩어리 전체 삭제
```

지울 것 ② (미션 6에서 넣은 것)

```
// 🧪 실험용 — 미션 7에서 지웁니다
$$('.searchBtn').forEach(btn => { … }); ← 이 덩어리 전체 삭제
```

🔰 **왜 지우나요?** 이제 이 일들을 **더 잘하는 기계**를 만들 거예요. 안 지우면 버튼이 **두 번씩** 반응해서 이상해집니다.

✍️ 2) ⑦번 버튼 연결 칸에 붙여넣기

```
/* 고른 학교를 담아 두는 상자 */
const PICKED = { A: null, B: null };

/* 학교 고르기 상자 하나를 작동시키는 기계.
   A상자와 B상자는 생김새가 똑같아서, 이 기계를 두 번만 부르면 됩니다! */
function setupPicker(slot) {
  const box     = $(`.picker[data-slot="${slot}"]`);
  const sidoSel = $('.sido', box);
  const input   = $('.q', box);
  const btn     = $('.searchBtn', box);
  const list    = $('.results', box);
  const chosen  = $('.chosen', box);

  /* 지역 목록 채우기 */
  sidoSel.innerHTML = SIDO_LIST
    .map(([code, name]) => `<option value="${code}">${name.replace('교육청', '')}
</option>`).join('');

  /* 학교 하나를 최종 선택했을 때 */
  function show(school) {
    PICKED[slot] = school;
    chosen.className = 'chosen on';
    chosen.innerHTML = `<strong>${esc(school.SCHUL_NM)}</strong>
      <small>${esc(school.SCHUL_KND_SC_NM || '')} · ${esc(school.ATPT_OFCDC_SC_NM || '')}
      · 학교코드 ${esc(school.SD_SCHUL_CODE)}</small>`;
    list.innerHTML = '';
    localStorage.setItem('owl' + slot, JSON.stringify(school));
  }

  /* 검색 버튼을 눌렀을 때 */
  async function doSearch() {
    const q = input.value.trim();
    if (q.length < 2) { setStatus('학교 이름을 <b>2글자 이상</b> 입력해 주세요.', 'warn'); return; }

    btn.disabled = true;                          // 두 번 눌리지 않게 잠그기
    setStatus(`‘${esc(q)}’ 검색 중…`, 'loading');
    try {
      const rows = await searchSchools(q, sidoSel.value);
      if (!rows.length) {
        list.innerHTML = '';
        setStatus(`‘${esc(q)}’(으)로 찾은 학교가 없습니다. 이름을 조금 줄여서 다시 찾아보세요.`,
'warn');
        return;
      }

      /* 찾은 학교들을 목록으로 만들기 */
      list.innerHTML = rows.map((r, i) => `<li data-i="${i}">
        ${esc(r.SCHUL_NM)}
        <small>${esc(r.SCHUL_KND_SC_NM || '')} · ${esc(r.LCTN_SC_NM || '')} ·
${esc(r.ATPT_OFCDC_SC_NM || '')}</small>
      </li>`).join('');

      /* 목록을 누르면 그 학교가 골라지게 하기 */
```

```
      $$('li', list).forEach(li =>
        li.addEventListener('click', () => show(rows[Number(li.dataset.i)])));

      const total = rows.totalCount;
      if (!getKey() && total > rows.length) {
        setStatus(`모두 <b>${total}곳</b> 중 <b>${rows.length}곳</b>만 보입니다.
          인증키가 없으면 한 번에 5줄까지만 받을 수 있습니다.
          아래 <b>🔑 NEIS 인증키 넣기</b>를 열어 키를 넣거나, 학교 이름을 더 자세히 입력해 보세요.`,
'warn');
      } else {
        setStatus(`${rows.length}곳을 찾았습니다. 목록에서 학교를 누르세요.`);
      }
    } catch (e) {
      showError(e);
    } finally {
      btn.disabled = false;                        // 꼭 다시 풀어 주기!
    }
  }

  btn.addEventListener('click', doSearch);
  input.addEventListener('keydown', e => { if (e.key === 'Enter') doSearch(); });

  /* 지난번에 골랐던 학교 되살리기 */
  const saved = localStorage.getItem('owl' + slot);
  if (saved) { try { show(JSON.parse(saved)); } catch (_) {} }
}
```

✍️ 3) ⑧번 시작 칸의 `★★★` 아래에 추가

```
  setupPicker('A');
  setupPicker('B');
```

💾 Ctrl+S → 🔄 F5

👀 확인!

1. 학교 A 칸에 `창원고` 를 넣고 **검색** → 아래에 **학교 목록이 상자로** 나타납니다.
2. 목록에서 `창원고등학교` **를 클릭** → 아래 칸이 이렇게 바뀝니다.

   ```
   창원고등학교
   고등학교 · 경상남도교육청 · 학교코드 9010114
   ```

3. 학교 B에도 **우리 학교 이름**을 넣고 골라 보세요.
4. **F5를 눌러도** 고른 학교가 그대로 남아 있습니다! (신기하죠?)

▢ **성공!** 학교 두 곳을 골랐어요! 🎉

> 🔰 **코드 뜯어보기**
>
> ① **왜 함수 하나로 A와 B를 둘 다 처리하나요?** A상자와 B상자는 **생김새가 똑같습니다.** 코드를 두 번 쓰면 고칠 때도 두 번 고쳐야 해요. 그래서 **기계를 하나만 만들고 두 번 부릅니다.** 이게 좋은 프로그램의 기본입니다!
>
> ```
> setupPicker('A');   // A상자 작동
> setupPicker('B');   // B상자 작동
> ```
>
> ② `$('.q', box)` **에서** `box` **는?** "**box 안에서만** class가 q인 걸 찾아라" 라는 뜻이에요. 이렇게 해야 A를 검색할 때 B의 입력칸을 잘못 읽지 않습니다.
>
> ③ `data-i="${i}"` **는 번호표** `<li>` 마다 "너는 몇 번째야"라는 **번호표**를 붙입니다. 나중에 클릭했을 때 `li.dataset.i` 로 번호를 꺼내 `rows[번호]` 로 그 학교를 찾아요.
>
> ④ `localStorage` 브라우저에 **작게 저장해 두는 서랍**입니다. F5를 눌러도 안 지워져요. `JSON.stringify` = 자료를 글자로 만들기 / `JSON.parse` = 글자를 자료로 되돌리기 (서랍에는 글자만 넣을 수 있어서 이렇게 바꿔 줍니다)
>
> ⑤ `finally` 성공하든 실패하든 **무조건** 마지막에 실행됩니다. 버튼 잠금을 꼭 풀어야 하니까요.
>
> ⑥ `btn.disabled = true` 버튼을 잠급니다. 사람들이 조급해서 여러 번 누르는 걸 막아 줘요.

✅ 미션 8. 인증키 넣기

무엇을 하나요? 검색 결과를 5곳 → 100곳까지 볼 수 있게 합니다.

✍️ 1) ⑦번 버튼 연결 칸 맨 아래에 붙여넣기

```
/* 인증키를 저장하고 지우는 기계 */
function setupKeyBox() {
  const input = $('#apiKey'), state = $('#keyState');
  const refresh = () => {
    const k = getKey();
    state.textContent = k
      ? `저장됨 (…${k.slice(-6)}) — 검색 결과를 100곳까지 볼 수 있습니다.`
      : '저장된 키 없음 (키 없이 사용 중 — 검색 결과 5곳 제한)';
  };
  $('#saveKeyBtn').addEventListener('click', () => {
    const v = input.value.trim();
    if (!v) { setStatus('인증키를 붙여넣어 주세요.', 'warn'); return; }
    localStorage.setItem('neisKey', v);
    input.value = '';
    refresh();
    setStatus('인증키를 이 컴퓨터에 저장했습니다.');
  });
  $('#clearKeyBtn').addEventListener('click', () => {
    localStorage.removeItem('neisKey');
    refresh();
    setStatus('인증키를 지웠습니다. 키 없이 사용합니다.');
  });
  refresh();
}
```

✍️ 2) ⑧번 시작 칸의 `★★★` 아래에 추가

```
  setupKeyBox();
```

💾 **Ctrl+S** → 🔄 **F5**

👀 **확인!**

1. 화면 아래쪽 `🔑 NEIS 인증키 넣기` 를 클릭해 펼칩니다.
2. 맨 아래에 **"저장된 키 없음 (키 없이 사용 중 — 검색 결과 5곳 제한)"** 이 보입니다.
3. 선생님이 알려 주신 인증키를 붙여넣고 **저장** → **"저장됨 (…abc123)"** 으로 바뀝니다.
4. 이제 `초등학교` 처럼 흔한 이름으로 검색하면 **훨씬 많이** 나옵니다!

☐ **성공!** 인증키가 저장됐어요.

> 🔰 **왜 키가 필요한가요?**
>
> NEIS는 누구나 쓸 수 있게 열어 두었지만, 너무 많이 쓰면 서버가 힘들어요. 그래서 **키 없이 쓰면 한 번에 5줄까지만** 줍니다. 급식 조회는 하루에 1~3줄이라 **키 없이도 충분**하고, **학교 검색만** 아쉬운 거예요.
>
> 키는 https://open.neis.go.kr 에서 회원가입하면 **무료로 바로** 받을 수 있습니다. 저장된 키는 **내 컴퓨터에만** 있고 인터넷으로 보내지 않아요.

🎊 2장 완성!

🆘 막혔다면 → `steps/2단계_app.js` 로 갈아타고 3장으로 오세요!

🍙 3장. 급식 불러오기

드디어 **진짜 급식**이 나옵니다! 이 장이 제일 재미있어요.

✅ 미션 9. 우리 학교 급식을 F12 창에 띄우기

✍️ 1) ①번 도구 칸 맨 아래에 붙여넣기

```
/* 날짜 모양 바꾸기 */
const toYmd   = s => String(s).replace(/-/g, '');                      // 2026-07-08 → 20260708
const fromYmd = s => `${s.slice(0, 4)}-${s.slice(4, 6)}-${s.slice(6, 8)}`; // 20260708 → 2026-07-08

/* 숫자 다듬기 */
const round1 = n => Math.round(n * 10) / 10;              // 697.4999 → 697.5
const clamp  = (n, lo, hi) => Math.min(hi, Math.max(lo, n)); // 값을 lo~hi 안에 가두기
```

✍️ 2) ③번 NEIS 부르기 칸 맨 아래에 붙여넣기

```
/* 학교 한 곳의 하루 급식 가져오기 */
function fetchMeal(school, ymd) {
  return callNeis('mealServiceDietInfo', {
    ATPT_OFCDC_SC_CODE: school.ATPT_OFCDC_SC_CODE,   // 어느 교육청
    SD_SCHUL_CODE: school.SD_SCHUL_CODE,             // 어느 학교
    MLSV_YMD: ymd,                                   // 어느 날
  });
}
```

✍️ 3) ⑦번 버튼 연결 칸 맨 아래에 붙여넣기

```
/* 🍼 임시 버전 — 미션 12에서 완성합니다 */
async function compare() {
  const A = PICKED.A, B = PICKED.B;
  if (!A || !B) { setStatus('학교 <b>A와 B를 모두</b> 골라 주세요.', 'warn'); return; }

  const ymd = toYmd($('#date').value);
  setStatus('급식 정보를 불러오는 중…', 'loading');
  try {
    const rowsA = await fetchMeal(A, ymd);
    const rowsB = await fetchMeal(B, ymd);
    console.log('학교 A 급식 원본 👉', rowsA);
    console.log('학교 B 급식 원본 👉', rowsB);
    setStatus(`A ${rowsA.length}줄, B ${rowsB.length}줄을 받았습니다!
               <b>F12 → Console</b> 을 열어 보세요. 🔍`);
  } catch (e) { showError(e); }
}
```

✍️ 4) ⑧번 시작 칸에서 미션 4의 🍼 실험용 버튼 코드를 지우고, 대신 추가

```
$('#compareBtn').addEventListener('click', compare);
```

💾 **Ctrl+S** → 🔄 **F5**

👀 **확인!**

1. 날짜를 **학기 중 평일**로 바꿉니다. (예: `2026-07-08`) > ⚠️ 주말·공휴일·방학에는 급식이 없어요!
2. 📊 **두 학교 비교하기** 를 누릅니다.
3. **F12 → Console** 에서 `학교 A 급식 원본 👉 ▶ Array(1)` 의 **▶ 를 펼치고**, 또 `0:` 을 펼칩니다.
4. 이런 게 보입니다! 👇

```
CAL_INFO: "697.5 Kcal"
DDISH_NM: "통밀밥 (6)<br/>순두부찌개 (1.5.9.13)<br/>브로콜리깨소스무침 (5.6.13)<br/>…"
NTR_INFO: "탄수화물(g) : 87.3<br/>단백질(g) : 40.4<br/>…"
```

☐ **성공!** 진짜 급식 자료를 가져왔어요! 🎉🎉🎉

🔰 자료를 보니 문제가 3개 보이죠?

```
DDISH_NM: "통밀밥 (6)<br/>순두부찌개 (1.5.9.13)<br/>사과 <br/>우유 (2)"
```

문제	어떻게 해결할까
① 요리가 전부 **한 줄로 붙어** 있다	` ` 를 기준으로 **잘라야** 한다
② 알레르기 번호가 **괄호 속에 섞여** 있다	괄호를 **떼어내야** 한다
③ 열량이 `"697.5 Kcal"` 라는 **글자**다	**숫자만 뽑아야** 계산할 수 있다

이렇게 **긴 글자를 쓸 수 있는 자료로 바꾸는 일**을 **파싱(parsing)** 이라고 합니다. 다음 미션에서 해요!

✅ 미션 10. 긴 글자를 쪼개기 (파싱)

🖍️ 먼저 손으로 해보기 (2분)

컴퓨터에게 시키기 전에 **내가 먼저** 해봅시다. `통밀밥 (6)<br/>순두부찌개 (1.5.9.13)<br/>사과` 를 표로 정리해 보세요.

요리 이름	알레르기 번호

어떤 순서로 했나요? ① `<br/>` 로 자르고 → ② 괄호 안 숫자를 떼고 → ③ 표에 넣었죠? **컴퓨터에게도 똑같이 시킵니다!**

✍️ ④번 글자 쪼개기 칸에 붙여넣기

```
/* '<br/>' 로 이어진 글자를 배열로 쪼개기 */
function splitBr(text) {
  return String(text || '')
    .split(/<br\s*\/?>/i)      // <br>, <br/>, <BR /> 전부 처리
    .map(s => s.trim())        // 앞뒤 공백 지우기
    .filter(Boolean);          // 빈 칸 버리기
}

/* "순두부찌개 (1.5.9.13)" → { name:'순두부찌개', allergens:[1,5,9,13] } */
function parseDishes(ddish) {
  return splitBr(ddish).map(raw => {
    let s = raw.replace(/\*/g, '').trim();        // 맨 앞의 * 지우기

    let allergens = [];
    const m = s.match(/\(([\d.\s]+)\)\s*$/);      // 맨 끝의 (숫자.숫자) 찾기
    if (m) {
      allergens = m[1].split('.').map(v => parseInt(v.trim(), 10)).filter(Number.isFinite);
      s = s.slice(0, m.index).trim();             // 괄호 앞까지만 남기기
    }
    // (자), (완), (과) 같은 학교별 표시도 지우기
    const name = s.replace(/\((자|완|과|꼴함유|[^)]{1,6})\)\s*$/, '').trim() || s;

    return { name, allergens };
  });
}

/* "697.5 Kcal" → 697.5 (숫자만 뽑기) */
function parseCalorie(cal) {
  const m = String(cal || '').match(/[\d.]+/);
  return m ? parseFloat(m[0]) : null;
}

/* "탄수화물(g) : 87.3" → { 탄수화물: { value:87.3, unit:'g' } } */
function parseNutrients(ntr) {
  const out = {};
  for (const line of splitBr(ntr)) {
    const m = line.match(/^(.+?)\s*\(([^)]*)\)\s*:\s*([\d.]+)/);
    if (!m) continue;
    const name = m[1].trim();                       // 비타민A
    out[name] = { value: parseFloat(m[3]), unit: m[2].trim() };
  }
  return out;
}

/* "쌀 : 국내산" → { 쌀: '국내산' } */
function parseOrigin(orplc) {
  const out = {};
  for (const line of splitBr(orplc)) {
    const i = line.indexOf(':');
    if (i < 0) continue;
    const k = line.slice(0, i).trim();
    const v = line.slice(i + 1).trim();
    if (k && v) out[k] = v;
```

```
  }
  return out;
}
```

✍️ `compare` 함수 안, `console.log('학교 B 급식 원본 …')` 아래에 한 줄 추가

```
    console.log('쪼갠 요리 👉', parseDishes(rowsA[0].DDISH_NM));
```

💾 **Ctrl+S** → 🔄 **F5** → **비교하기** 버튼

👀 **확인!** F12 → Console 에서 `쪼갠 요리 👉 ▶ Array(7)` 을 펼쳐 보세요.

```
0: {name: "통밀밥", allergens: [6]}
1: {name: "순두부찌개", allergens: [1, 5, 9, 13]}
2: {name: "브로콜리깨소스무침", allergens: [5, 6, 13]}
…
```

한 덩어리 글자가 깔끔한 표로 바뀌었습니다!

▢ **성공!** 글자를 쪼갰어요! 🎉

🔰 정규식(`/…/`)이 뭐예요?

"이런 모양의 글자를 찾아 줘" 라고 말하는 방법이에요. 지금은 이 4개만 알면 충분합니다.

기호	뜻	예
`\d`	숫자 하나	`\d\d` → `13`
`+`	앞의 것이 1개 이상	`\d+` → `1358`
`$`	글자의 **맨 끝**	`\(\d+\)$` → 끝에 있는 `(13)`
`\.`	진짜 점(.)	`1.5` 의 점

`/\(([\d.\s]+)\)\s*$/` 는 → **"맨 끝에 있는, 숫자와 점으로만 된 괄호"** 를 찾으라는 뜻!

😌 **지금 다 이해 못 해도 괜찮아요.** 그대로 써도 잘 작동합니다. 나중에 필요할 때 다시 보면 됩니다!

`.split()` / `.trim()` / `.filter()`

이름	하는 일
`split('구분자')`	구분자를 기준으로 **자르기**
`trim()`	앞뒤 **공백 지우기**
`filter(Boolean)`	**빈 것 버리기**

✅ 미션 11. 자료를 정리해서 상자에 담기

무엇을 하나요? 쪼갠 자료들을 `R` 이라는 **상자 하나**에 정리해 담습니다.

✍️ 1) ②번 자료 칸 맨 아래에 붙여넣기

```
/* ⚠️ 영양 참고 기준값 (1인 1회 중식 기준)
   학교급식법 시행규칙 별표 3 「학교급식의 영양관리기준」을 참고한 값입니다.
   기준이 바뀔 수 있으니, 최신 값을 찾아 이 숫자만 고치면 됩니다. */
const STANDARD = {
  e13: { label: '초등 1~3학년', 에너지: 534, 단백질: 8.4,  비타민A: 97,  티아민: 0.20, 리보플라빈:
0.24, 비타민C: 13.4, 칼슘: 184, 철분: 2.4 },
  e46: { label: '초등 4~6학년', 에너지: 634, 단백질: 11.7, 비타민A: 127, 티아민: 0.24, 리보플라빈:
0.30, 비타민C: 19.6, 칼슘: 217, 철분: 3.0 },
  m:   { label: '중학교',       에너지: 800, 단백질: 16.6, 비타민A: 167, 티아민: 0.34, 리보플라빈:
0.43, 비타민C: 26.7, 칼슘: 267, 철분: 4.5 },
  h:   { label: '고등학교',     에너지: 900, 단백질: 20.0, 비타민A: 200, 티아민: 0.40, 리보플라빈:
0.50, 비타민C: 33.4, 칼슘: 300, 철분: 5.4 },
};

/* 학교급 이름 → 어떤 기준을 쓸지 정하기 */
function levelOf(schoolKind) {
  if (!schoolKind) return 'm';
  if (schoolKind.includes('초등')) return 'e46';   // 학년을 알 수 없어 4~6학년 기준
  if (schoolKind.includes('중'))   return 'm';
  if (schoolKind.includes('고'))   return 'h';
  return 'm';
}
```

✍️ 2) ⑤번 계산하기 칸에 붙여넣기

```
/* 급식 한 끼를 분석해서 정리된 상자 R 을 만들어 돌려줍니다. */
function analyze(meal, school, levelKey) {
  const R = {
    school, meal, levelKey,
    std: STANDARD[levelKey],
    mealName: meal.MMEAL_SC_NM,
    date: meal.MLSV_YMD,
    servings: Number(meal.MLSV_FGR) || null,
  };

  /* 긴 글자를 쓸 수 있는 자료로 바꿔 담기 */
  R.dishes = parseDishes(meal.DDISH_NM);
  R.kcal   = parseCalorie(meal.CAL_INFO);
  R.origin = parseOrigin(meal.ORPLC_INFO);
  R.nutri  = parseNutrients(meal.NTR_INFO);
  if (R.kcal != null) R.nutri['에너지'] = { value: R.kcal, unit: 'kcal' };

  /* ★★★ 앞으로 여기에 계속 추가합니다 ★★★ */

  return R;
}
```

✍️ 3) `compare` 함수 안, 아까 넣은 `console.log('쪼갠 요리 …')` 아래에 추가

```
    console.log('정리한 상자 R 👉', analyze(rowsA[0], A, levelOf(A.SCHUL_KND_SC_NM)));
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 **확인!** Console 에서 `정리한 상자 R 👉 ▶ {…}` 를 펼쳐 보세요.

```
dishes: ▶ Array(7)          ← 쪼갠 요리들
kcal: 697.5                 ← 숫자로 바뀐 열량!
nutri: ▶ {탄수화물: {…}, 단백질: {…}, …}
origin: ▶ {쇠고기(종류): "국내산(육우)", …}
servings: 582
std: ▶ {label: "초등 4~6학년", 에너지: 634, …}
```

흩어져 있던 자료가 상자 하나에 예쁘게 정리됐습니다!

▢ **성공!** R 상자가 만들어졌어요.

🔰 왜 상자 `R` 에 담나요?

자료가 따로따로 있으면 들고 다니기 불편해요.

```
const R = {};        // 빈 상자를 만들고
R.dishes = …;        // 하나씩 넣는다
R.kcal   = …;
```

이렇게 하면 나중에 `R.kcal`, `R.dishes` 처럼 **점(.)** 으로 꺼내 쓸 수 있어요. **앞으로 미션 17, 18에서 이 상자에 계속 물건을 더 넣을 겁니다.** 그래서 `★★★` 표시를 해 둔 거예요!

`levelOf(A.SCHUL_KND_SC_NM)` 학교급이 `'초등학교'` 면 `'e46'`, `'고등학교'` 면 `'h'` 를 돌려줍니다. 초등학교는 몇 학년인지 NEIS가 안 알려 줘서 **4~6학년 기준**을 씁니다.

⭐ 미션 12. 화면에 식단 띄우기 — 가장 큰 순간!

무엇을 하나요? 드디어 **화면에 진짜 식단**이 나옵니다!

✍️ 1) ⑥번 화면 그리기 칸에 붙여넣기

```
/* 지금 화면에 그려져 있는 결과를 기억해 두는 상자 */
let CURRENT = null;

/* 요리 한 줄 만들기 */
function dishLine(d) {
  return `<li>
    <span class="dname">${esc(d.name)}</span>
  </li>`;
}

/* 카드 아래쪽의 큰 숫자 상자 */
function factsBox(R) {
  return `<div class="facts">
    <div class="fact"><b>${R.kcal == null ? '-' : round1(R.kcal)}</b><span>kcal</span></div>
    <div class="fact"><b>${R.dishes.length}</b><span>요리 수</span></div>
    <div class="fact"><b>${R.servings ? R.servings.toLocaleString() : '-'}</b><span>급식 인원</span>
</div>
  </div>`;
}

/* 식단 카드 2장 만들기 */
function renderMealCards(A, B) {
  const card = (R, cls) => `<div class="meal-card ${cls}">
    <h3><span class="chip chip-${cls}">학교 ${cls.toUpperCase()}</span> ${esc(R.school.SCHUL_NM)}
      <small>${esc(R.school.ATPT_OFCDC_SC_NM || '')} · ${esc(R.school.LCTN_SC_NM || '')}</small>
</h3>
    <ul class="dish-list">${R.dishes.map(dishLine).join('') || '<li>식단 정보가 없습니다.</li>'}</ul>
    ${factsBox(R)}
  </div>`;

  $('#mealCards').innerHTML = card(A, 'a') + card(B, 'b');
}

/* 결과 전체를 그리는 기계 */
function renderAll(A, B) {
  CURRENT = { A, B };
  renderMealCards(A, B);

  /* ★★★ 앞으로 여기에 계속 한 줄씩 추가합니다 ★★★ */

  $('#result').hidden = false;      // 숨겨 두었던 결과 영역을 보이게!
}
```

✍️ 2) ⑦번 칸의 `compare` 함수를 통째로 지우고, 아래 것으로 바꾸기

```
/* 두 학교 비교하기 — 완성 버전! */
async function compare() {
  const A = PICKED.A, B = PICKED.B;
  if (!A || !B) { setStatus('학교 <b>A와 B를 모두</b> 골라 주세요.', 'warn'); return; }
  if (A.SD_SCHUL_CODE === B.SD_SCHUL_CODE) {
    setStatus('같은 학교를 두 번 골랐습니다. 서로 <b>다른 학교</b>를 골라 주세요.', 'warn'); return;
  }

  const ymd  = toYmd($('#date').value);
  const type = $('#mealType').value;          // '1' 조식 / '2' 중식 / '3' 석식
  if (!ymd) { setStatus('날짜를 골라 주세요.', 'warn'); return; }

  $('#compareBtn').disabled = true;
  setStatus('급식 정보를 불러오는 중…', 'loading');

  try {
    /* 두 학교를 '동시에' 부르기 — 하나씩 기다리면 두 배 느립니다! */
    const [rowsA, rowsB] = await Promise.all([fetchMeal(A, ymd), fetchMeal(B, ymd)]);

    /* 하루에 조식·중식·석식이 다 올 수 있으니, 고른 끼니만 찾기 */
    const pick = rows => rows.find(r => r.MMEAL_SC_CODE === type) || null;
    let mA = pick(rowsA), mB = pick(rowsB);

    const missing = [];
    if (!mA) missing.push(A.SCHUL_NM);
    if (!mB) missing.push(B.SCHUL_NM);

    if (missing.length) {
      const other = rows => [...new Set(rows.map(r => r.MMEAL_SC_NM))].join('/');
      const hintA = rowsA.length ? `${A.SCHUL_NM}: ${other(rowsA)} 있음` : `${A.SCHUL_NM}: 급식 없음
`;
      const hintB = rowsB.length ? `${B.SCHUL_NM}: ${other(rowsB)} 있음` : `${B.SCHUL_NM}: 급식 없음
`;
      setStatus(`<b>${esc(missing.join(', '))}</b>의 ${fromYmd(ymd)} 자료가 없습니다.
        (${esc(hintA)} / ${esc(hintB)})<br>
        주말·공휴일·방학이거나 학교가 아직 등록하지 않은 날일 수 있습니다.
        <b>🔎 급식 있는 날 찾기</b>를 눌러 보세요.`, 'warn');
      return;
    }

    /* 학교급에 맞는 기준 고르기 */
    const lv = $('#levelSel').value;
    const keyA = lv === 'auto' ? levelOf(A.SCHUL_KND_SC_NM) : lv;
    const keyB = lv === 'auto' ? levelOf(B.SCHUL_KND_SC_NM) : lv;

    renderAll(analyze(mA, A, keyA), analyze(mB, B, keyB));
    setStatus('');
    $('#result').scrollIntoView({ behavior: 'smooth', block: 'start' });
  } catch (e) {
    showError(e);
  } finally {
    $('#compareBtn').disabled = false;
```

```
  }
}
```

💾 **Ctrl+S** → 🔄 **F5** → 학교 두 곳 고르고 → 날짜를 **학기 중 평일**로 → **비교하기**

👀 **확인!** 화면이 스르륵 내려가면서 **식단 카드 2장이 나란히** 나타납니다!

```
┌ 학교 A  경기초등학교 ──────┐  ┌ 학교 B  창원고등학교 ──────┐
│  통밀밥                  │  │  백미밥                  │
│  순두부찌개              │  │  아라비아따파스타        │
│  브로콜리깨소스무침      │  │  단호박크림스프          │
│  …                       │  │  …                       │
│ ┌──────┬──────┬──────┐   │  │ ┌──────┬──────┬──────┐   │
│ │697.5 │  7   │ 582  │   │  │ │1158.7│  6   │ 630  │   │
│ │kcal  │요리 수│급식인원│   │  │ │ kcal │요리 수│급식인원│   │
└──────────────────────────┘  └──────────────────────────┘
```

▢ **성공!!** 진짜 급식이 화면에 나왔어요! 🎉🎉🎉 **친구에게 자랑하세요!**

🔰 코드 뜯어보기

① `Promise.all` — **두 배 빠르게!** ⚡

```
하나씩:      A학교 부르기(1초) → 끝나면 B학교 부르기(1초)  = 2초
Promise.all: A와 B를 동시에 부르기                        = 1초  ⚡
```

② `const [rowsA, rowsB] = await Promise.all([...])` 결과 2개를 한 번에 두 상자에 나눠 담는 방법이에요. (구조 분해라고 불러요)

③ `rows.find(r => r.MMEAL_SC_CODE === type)` `find` 는 **조건에 맞는 첫 번째 것 하나**를 찾아 줍니다. `===` 는 **"똑같다"** 는 뜻이에요. (`=` 은 "넣는다", `===` 은 "같다" — 다릅니다!)

④ `$('#result').hidden = false;` 결과 영역은 원래 **숨겨져** 있었어요. 이제 보이게 하는 겁니다.

⑤ `scrollIntoView` 화면을 결과가 있는 곳으로 **부드럽게 스크롤**해 줍니다.

⑥ `card(A, 'a') + card(B, 'b')` 카드 만드는 기계를 **두 번 불러서** 이어 붙입니다. A는 파란색, B는 주황색이에요.

✅ 미션 13. 알레르기 번호 붙이기

✍️ 1) ②번 자료 칸 맨 아래에 붙여넣기

```
/* 알레르기 유발 식품 번호 (식품위생법에서 정한 것) */
const ALLERGEN = {
  1: '난류(계란)', 2: '우유', 3: '메밀', 4: '땅콩', 5: '대두(콩)', 6: '밀',
  7: '고등어', 8: '게', 9: '새우', 10: '돼지고기', 11: '복숭아', 12: '토마토',
  13: '아황산류', 14: '호두', 15: '닭고기', 16: '쇠고기', 17: '오징어',
  18: '조개류(굴·전복·홍합 등)', 19: '잣',
};
```

✍️ 2) ⑥번 칸의 `dishLine` 함수를 통째로 지우고, 아래 것으로 바꾸기

```
/* 요리 한 줄 만들기 (알레르기 번호 추가!) */
function dishLine(d) {
  return `<li>
    <span class="dname">${esc(d.name)}</span>
    <span class="algs">${d.allergens.map(n =>
      `<span class="alg" title="${esc(ALLERGEN[n] || '기타')}">${n}</span>`).join('')}</span>
  </li>`;
}
```

✍️ 3) ⑥번 칸의 `renderMealCards` 함수도 통째로 지우고, 아래 것으로 바꾸기

카드 맨 아래에 **오늘 나온 알레르기 번호를 모아** 한 줄로 적어 줍니다.

```
/* 식단 카드 2장 만들기 (알레르기 번호 모음 추가!) */
function renderMealCards(A, B) {
  const card = (R, cls) => {
    /* 이 식단에 나온 알레르기 번호를 모두 모아 중복 없이 정렬 */
    const allAlg = [...new Set(R.dishes.flatMap(d => d.allergens))].sort((a, b) => a - b);

    return `<div class="meal-card ${cls}">
      <h3><span class="chip chip-${cls}">학교 ${cls.toUpperCase()}</span> ${esc(R.school.SCHUL_NM)}
        <small>${esc(R.school.ATPT_OFCDC_SC_NM || '')} · ${esc(R.school.LCTN_SC_NM || '')}</small>
</h3>
      <ul class="dish-list">${R.dishes.map(dishLine).join('') || '<li>식단 정보가 없습니다.</li>'}
</ul>
      ${factsBox(R)}
      <p style="font-size:12px;color:var(--muted);margin:12px 0 0">
        알레르기 번호 모음: ${allAlg.length
          ? allAlg.map(n => `${n}(${esc(ALLERGEN[n] || '?')})`).join(', ')
          : '표시 없음'}
      </p>
    </div>`;
  };
  $('#mealCards').innerHTML = card(A, 'a') + card(B, 'b');
}
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 확인!

1. 요리 이름 **오른쪽에 주황색 동그란 숫자**가 붙었습니다. 그 숫자에 **마우스를 올려 보세요** → `난류(계란)`, `대두(콩)` 같은 이름이 뜹니다!
2. 카드 맨 아래에 이런 줄이 생겼습니다.

   ```
   알레르기 번호 모음: 1(난류(계란)), 2(우유), 5(대두(콩)), 6(밀), 9(새우) …
   ```

▢ **성공!** 알레르기 번호가 나왔어요.

🔰 `flatMap` 과 `Set` 이 뭐죠?

```
R.dishes.flatMap(d => d.allergens)
  요리마다 [6], [1,5,9,13], [5,6,13] … 을 꺼내서 → [6,1,5,9,13,5,6,13,…] 로 납작하게 펼침
          ↓ new Set(…)  = 중복 없애기
  [6,1,5,9,13]
          ↓ .sort((a,b) => a-b)  = 작은 수부터 줄 세우기
  [1,5,6,9,13]
```

`flat` 은 **납작하게**라는 뜻이에요. 배열 안의 배열을 하나로 펼쳐 줍니다.

🔰 코드 뜯어보기

`title="…"` → 마우스를 올렸을 때 뜨는 **말풍선**을 만드는 HTML 표시입니다.

`ALLERGEN[n] || '기타'` `||` 는 **"앞의 것이 없으면 뒤의 것"** 이라는 뜻. 번호표에 없는 숫자가 오면 `'기타'` 라고 보여 줍니다. (**안전장치!**)

`map` 안에 또 `map` 이 있어요!

- 바깥 `map` → **요리마다** `<li>` 를 만든다
- 안쪽 `map` → **알레르기 번호마다** 동그라미를 만든다

✅ 미션 14. 원산지 표 만들기

✍️ 1) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 원산지 비교 표 만들기 */
function renderOrigin(A, B) {
  const keys = [...new Set([...Object.keys(A.origin), ...Object.keys(B.origin)])];
  if (!keys.length) {
    $('#originTable').innerHTML = '<tr><td>원산지 정보가 제공되지 않았습니다.</td></tr>';
    return;
  }
  const rows = keys.map(k => {
    const a = A.origin[k] || '-', b = B.origin[k] || '-';
    const cls = (a !== b) ? 'diff' : 'same';        // 다르면 진하게!
    return `<tr><th style="width:30%">${esc(k)}</th>
      <td class="${cls}">${esc(a)}</td><td class="${cls}">${esc(b)}</td></tr>`;
  }).join('');

  $('#originTable').innerHTML =
    `<thead><tr><th>품목</th><th>${esc(A.school.SCHUL_NM)}</th><th>${esc(B.school.SCHUL_NM)}</th>
</tr></thead>
     <tbody>${rows}</tbody>`;
}
```

✍️ 2) `renderAll` 함수 안의 `★★★` 아래에 한 줄 추가

```
renderOrigin(A, B);
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 **확인!** 아래로 스크롤하면 🌏 **원산지 비교** 표가 보입니다. 두 학교가 **다르게** 적은 항목은 **진하게** 표시돼요! (예: 명태 — 러시아산 vs 국내산)

▢ **성공!** 원산지 표가 나왔어요.

> 🔰 **코드 뜯어보기**
>
> `Object.keys(A.origin)` → 상자의 **이름표만** 모아 배열로 만듭니다. `{쌀:'국내산', 배추:'국내산'}` → `['쌀', '배추']`
>
> `new Set([...])` → **중복을 없애 주는** 상자입니다. A와 B의 품목을 합치면 겹치는 게 많으니, Set으로 한 번씩만 남깁니다.
>
> `...` **(점 세 개, 스프레드)** → 배열을 **펼치는** 기호예요. `[...배열1, ...배열2]` → 두 배열을 하나로 합칩니다.
>
> `(a !== b) ? 'diff' : 'same'` `조건 ? 참일때 : 거짓일때` — `if` 를 한 줄로 줄인 것입니다. (삼항 연산자) "a와 b가 다르면 `diff`, 같으면 `same`"

🎊 3장 완성! 절반을 넘었어요!

> 🆘 막혔다면 → `steps/3단계_app.js` 로 갈아타고 4장으로!

📊 4장. 그래프 그리기

그래프 프로그램 없이, **오직 계산만으로** 그래프를 그립니다!

⭐ 미션 15. 영양소 막대그래프 그리기

💡 먼저 원리부터 (초등 산수예요!)

$$\text{막대 길이(\%)} = \frac{\text{내 값}}{\text{가장 큰 값}} \times 100$$

예제 — 칼슘: A학교 265.8mg, B학교 154.3mg → 가장 큰 값은 **265.8**

- A: 265.8 ÷ 265.8 × 100 = **100%**
- B: 154.3 ÷ 265.8 × 100 = **58%**

이 숫자를 CSS의 `width`(너비)에 넣으면 **그게 바로 막대그래프**입니다! 🎉

```
<div style="width:100%">A학교</div>     ← 긴 막대
<div style="width:58%">B학교</div>      ← 짧은 막대
```

🖍️ 직접 계산해 보기

철분이 A학교 2.8mg, B학교 4.5mg 일 때 가장 큰 값 = _____ → A = _____% → B = _____%

✍️ 1) ②번 자료 칸 맨 아래에 붙여넣기

```
/* 화면에 보여 줄 영양소 순서 */
const NUTRI_ORDER = ['에너지', '탄수화물', '단백질', '지방', '칼슘', '철분',
  '비타민A', '비타민C', '티아민', '리보플라빈'];
```

✍️ 2) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 색 설명(범례) */
function renderLegend(A, B) {
  const html = `
    <span><i style="background:var(--a)"></i>${esc(A.school.SCHUL_NM)}</span>
    <span><i style="background:var(--b)"></i>${esc(B.school.SCHUL_NM)}</span>`;
  $('#legend1').innerHTML = html;
  $('#legend2').innerHTML = html;
}

/* 막대 한 줄 만들기 (A와 B를 위아래로 나란히) */
function barRow(label, sub, aVal, bVal, max, unit, goal) {
  const bar = (v, cls) => {
    if (v == null) return `<div class="bar-track"><div class="bar-fill ${cls} low" style="width:0%">
자료 없음</div></div>`;
    const pct = max > 0 ? clamp((v / max) * 100, 0, 100) : 0;      // ★ 여기가 계산!
    const text = `${round1(v)}${unit || ''}`;
    return `<div class="bar-track">
      <div class="bar-fill ${cls}${pct < 30 ? ' low' : ''}" style="width:${pct.toFixed(1)}%">${text}
</div>
    </div>`;
  };
  /* 권장 기준값 자리에 세로선 하나 세우기 */
  const goalMark = (goal != null && max > 0 && goal <= max)
    ? `<div class="bar-goal" style="left:${((goal / max) * 100).toFixed(1)}%"></div>` : '';

  return `<div class="bar-item">
    <div class="bar-label">${esc(label)}${sub ? `<small>${esc(sub)}</small>` : ''}</div>
    <div class="bar-track-set">${goalMark}${bar(aVal, 'a')}${bar(bVal, 'b')}</div>
  </div>`;
}

/* 영양소 막대 그리기 */
function renderNutriBars(A, B) {
  const rows = NUTRI_ORDER.map(name => {
    const a = A.nutri[name]?.value ?? null;
    const b = B.nutri[name]?.value ?? null;
    if (a == null && b == null) return '';            // 둘 다 없으면 줄을 안 만듦

    const unit = A.nutri[name]?.unit || B.nutri[name]?.unit || '';
    const gA = A.std[name], gB = B.std[name];
    /* 두 학교의 기준이 같을 때만 기준선을 그립니다 */
    const goal = (gA != null && gA === gB) ? gA : null;

    const max = Math.max(a ?? 0, b ?? 0, goal ?? 0) * 1.12 || 1;
    const sub = (gA != null || gB != null)
      ? `기준 A ${gA ?? '-'} / B ${gB ?? '-'}`
      : '';
    return barRow(`${name}${unit ? ` (${unit})` : ''}`, sub, a, b, max, '', goal);
  }).join('');

  $('#nutriBars').innerHTML = rows || '<p>영양 정보가 제공되지 않았습니다.</p>';

  $('#nutriHint').innerHTML = A.levelKey === B.levelKey
```

```
    ? `막대 길이는 두 학교 중 큰 값에 맞춰 그렸습니다.
       세로선(<b>기준</b>)은 <b>${esc(STANDARD[A.levelKey].label)}</b> 1인 1회 참고 기준값입니다.`
    : `막대 길이는 두 학교 중 큰 값에 맞춰 그렸습니다.
       두 학교의 <b>학교급이 달라 적용 기준도 다르므로</b> 세로 기준선은 생략했습니다
       (A: ${esc(A.std.label)} / B: ${esc(B.std.label)}). 항목 이름 아래에 각각의 기준값을 적어 두었습
니다.`;
}
```

✍️ 3) `renderAll` 함수 안의 `★★★` 아래에 두 줄 추가

```
  renderLegend(A, B);
  renderNutriBars(A, B);
```

💾 **Ctrl+S** → 🔄 **F5** → **비교하기**

👀 **확인!** 🥗 **영양소 비교** 칸에 **파란색(A)과 주황색(B) 막대가 10줄** 그려집니다! 막대 안에 숫자도 적혀 있어요.

> **기준선(세로 검은 선)이 보이려면?** 두 학교의 **학교급이 같아야** 합니다. 초등학교와 고등학교를 비교하면 기준이 달라서 선이 안 나와요. 화면 위쪽 `영양 기준` 을 `고등학교` 로 고정하면 바로 나타납니다!

▢ **성공!** 막대그래프를 그렸어요! 🎉

> 🔰 **코드 뜯어보기**
>
> `A.nutri[name]?.value ?? null`
>
> - `?.` → 없으면 오류 내지 말고 넘어가
> - `??` → 앞이 없으면 뒤의 값 사용
>
> 합치면 **"있으면 값을, 없으면 null을"** 이라는 뜻이에요.
>
> `toFixed(1)` → 소수점 첫째 자리까지만 (58.333 → `58.3`)
>
> `* 1.12` → 가장 큰 값보다 **12% 여유**를 둡니다. 막대가 꽉 차면 답답해 보여서요.
>
> `pct < 30 ? ' low' : ''` 막대가 짧으면(30% 미만) 숫자가 안 보이니까, **글씨를 막대 밖에** 쓰라는 표시를 붙입니다.
>
> `Math.max(a, b, goal)` → 셋 중 **가장 큰 값**을 골라 줍니다.

✅ 미션 16. 요리를 식품군으로 자동 분류하기

무엇을 하나요? 요리 이름만 보고 컴퓨터가 **곡류·채소·과일…** 을 알아맞히게 합니다.

✍️ 1) ②번 자료 칸 맨 아래에 붙여넣기

> 낱말이 아주 많죠? **직접 타이핑하지 말고 꼭 복사해서** 붙여넣으세요!

```
/* 식품군 사전 — 요리 이름에 이 낱말이 들어 있으면 그 식품군으로 봅니다.
   규칙: "가장 긴 낱말이 이긴다" */
const FOOD_GROUPS = [
  {
    key: '단백질', label: '고기·생선·달걀·콩', color: '#c0504d',
    words: ['고기', '쇠고기', '소고기', '돼지', '돈육', '제육', '수육', '육개장', '갈비', '떡갈비',
      '불고기', '장조림', '산적', '육전', '닭', '치킨', '찜닭', '오리', '햄', '소시지', '비엔나',
      '베이컨', '스팸', '너겟', '너깃', '함박', '미트볼', '스테이크', '탕수', '탕수육', '까스',
      '가스', '커틀렛', '크로켓', '고로케', '깐풍', '유린', '강정', '멘보샤', '순대', '곱창',
      '생선', '고등어', '삼치', '갈치', '임연수', '코다리', '동태', '명태', '조기', '가자미',
      '오징어', '새우', '게살', '맛살', '꽃게', '참치', '멸치', '쭈꾸미', '주꾸미', '낙지', '문어',
      '어묵', '달걀', '계란', '메추리', '두부', '순두부', '유부', '완자', '콩', '콩자반',
      '목살', '삼겹', '등심', '안심', '채끝', '훈제', '돈까스', '롤까스'],
  },
  {
    key: '유제품', label: '우유·유제품', color: '#4f7bc0',
    words: ['우유', '흰우유', '초코우유', '바나나우유', '두유', '요구르트', '요거트', '요플레',
      '야쿠르트', '치즈', '체다', '모짜렐라', '리코타', '밀크', '라떼', '생크림', '크림',
      '버터', '푸딩', '아이스크림'],
  },
  {
    key: '과일', label: '과일류', color: '#d1607f',
    words: ['사과', '배', '귤', '감귤', '오렌지', '바나나', '포도', '청포도', '수박', '참외',
      '딸기', '토마토', '방울토마토', '키위', '멜론', '파인애플', '자두', '복숭아', '천도복숭아',
      '한라봉', '천혜향', '블루베리', '망고', '과일', '후르츠', '프루츠', '애플', '단감', '홍시',
      '대추', '살구', '체리', '레몬', '자몽', '무화과', '석류', '유자'],
  },
  {
    key: '채소', label: '채소류', color: '#4a9b5c',
    words: ['나물', '무침', '김치', '겉절이', '깍두기', '총각김치', '열무', '얼갈이', '샐러드',
      '생채', '숙채', '쌈', '야채', '채소', '브로콜리', '시금치', '콩나물', '숙주', '오이',
      '상추', '양상추', '배추', '양배추', '파프리카', '버섯', '가지', '애호박', '호박', '단호박',
      '도라지', '미역', '파래', '톳', '조미김', '김구이', '단무지', '피클', '장아찌', '부추',
      '시래기', '취나물', '참나물', '우엉', '연근', '당근', '죽순', '마늘', '양파', '대파',
      '쪽파', '셀러리', '샐러리', '콜라비', '비타민', '청경채', '두릅', '냉이', '달래', '곤드레'],
  },
  {
    key: '곡류', label: '곡류·주식', color: '#c9932f',
    words: ['밥', '백미', '흑미', '현미', '잡곡', '보리', '통밀', '기장', '죽', '국수', '면',
      '라면', '우동', '파스타', '스파게티', '라이스', '빵', '떡', '만두', '김밥', '주먹밥',
      '유부초밥', '덮밥', '국밥', '리조또', '필라프', '누룽지', '수제비', '떡국', '칼국수',
      '쫄면', '비빔면', '냉면', '짜장', '자장', '토스트', '시리얼', '또띠아', '부리또',
      '와플', '팬케이크', '마카로니', '감자', '고구마', '옥수수'],
  },
];

/* 위 다섯 군데 어디에도 안 들어가면 '기타' */
const GROUP_ETC = { key: '기타', label: '기타·후식·국물', color: '#8b93a8' };

/* 가공식품·튀김으로 볼 낱말 (완벽하지 않아요 — 나중에 함께 고쳐 봅시다!) */
const PROCESSED_WORDS = ['튀김', '튀기', '까스', '가스', '너겟', '너깃', '후라이', '프라이',
  '강정', '탕수', '소시지', '햄', '베이컨', '스팸', '어묵', '맛살', '크랩', '만두', '핫도그',
  '치킨', '크로켓', '고로케', '피자', '버거', '라면', '프랑크', '비엔나', '미트볼', '함박',
  '커틀렛', '유린기', '깐풍', '멘보샤', '통조림', '케이크', '쿠키', '도넛', '츄러스', '웨지'];
```

✍️ 2) ④번 글자 쪼개기 칸 맨 아래에 붙여넣기

```
/* 요리 이름 → 식품군 ("가장 긴 낱말이 이긴다") */
function classifyDish(name) {
  let best = GROUP_ETC, bestLen = 0;
  for (const g of FOOD_GROUPS) {
    for (const w of g.words) {
      if (name.includes(w) && w.length > bestLen) { bestLen = w.length; best = g; }
    }
  }
  return best;
}
```

✍️ 3) ④번 칸의 `parseDishes` 안에서 맨 아래 `return { name, allergens };` 를 찾아 아래처럼 바꾸기

```
    return {
      name,
      allergens,
      group: classifyDish(name),                              // 어느 식품군인가
      processed: PROCESSED_WORDS.some(w => name.includes(w)), // 가공/튀김인가
    };
```

✍️ 4) ⑥번 칸의 `dishLine` 함수를 통째로 지우고, 아래 것으로 바꾸기

```
/* 요리 한 줄 만들기 (식품군 이름표 + 가공/튀김 표시 추가!) */
function dishLine(d) {
  return `<li>
    <span class="gname" style="border-
color:${d.group.color};color:${d.group.color}">${esc(d.group.label.split('·')[0])}</span>
    <span class="dname">${esc(d.name)}</span>
    ${d.processed ? '<span class="flag" title="이름에 가공식품·튀김 낱말이 들어 있습니다">가공/튀김
</span>' : ''}
    <span class="algs">${d.allergens.map(n =>
      `<span class="alg" title="${esc(ALLERGEN[n] || '기타')}">${n}</span>`).join('')}</span>
  </li>`;
}
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 **확인!** 요리 이름 **왼쪽에 색깔 이름표**가 붙었습니다!

```
[곡류]  통밀밥                          6
[고기]  순두부찌개                  1 5 9 13
[채소류] 브로콜리깨소스무침           5 6 13
[고기]  허브치킨플레로티 [가공/튀김]  1 2 5 6 …
[채소류] 배추김치                       9
[과일류] 사과
[우유]  우유                            2
```

▢ **성공!** 컴퓨터가 요리를 분류했어요! 🎉

> 🔰 **"가장 긴 낱말이 이긴다"는 왜 필요할까?**
>
> `배추김치` 에는 `배`**(과일)** 도 있고 `배추`**(채소)** 도 들어 있어요! 그냥 먼저 찾은 걸 쓰면 배추김치가 **과일**이 되어 버립니다. 😱
>
> 그래서 **더 긴 낱말**을 믿습니다. `배추`(2글자) > `배`(1글자) → **채소류** ✅
>
> ```
> if (name.includes(w) && w.length > bestLen) { bestLen = w.length; best = g; }
>                         └── 지금까지 찾은 것보다 더 길면 → 바꾼다
> ```
>
> `for (const g of FOOD_GROUPS)` → 식품군 5개를 하나씩 꺼내 봅니다. `name.includes(w)` → "이름 안에 이 낱말이 들어 있니?" (있으면 true) `.some(w => …)` → "하나라도 해당되면 true"

✅ 미션 17. 식품군 색깔 띠 만들기 + 🔍 컴퓨터의 실수 찾기

✍️ 1) `analyze` 함수 안의 `★★★` 아래에 붙여넣기

```
  /* 식품군별로 요리가 몇 가지인지 세기 */
  R.groupCount = {};
  for (const g of [...FOOD_GROUPS, GROUP_ETC]) R.groupCount[g.key] = 0;
  for (const d of R.dishes) R.groupCount[d.group.key]++;
  R.groupsCovered = FOOD_GROUPS.filter(g => R.groupCount[g.key] > 0).length;   // 0~5
  R.processedCnt  = R.dishes.filter(d => d.processed).length;
```

✍️ 2) ⑥번 칸의 `factsBox` 함수를 통째로 지우고, 아래 것으로 바꾸기

```
/* 큰 숫자 상자 4개 (식품군 칸 추가!) */
function factsBox(R) {
  return `<div class="facts">
    <div class="fact"><b>${R.kcal == null ? '-' : round1(R.kcal)}</b><span>kcal</span></div>
    <div class="fact"><b>${R.dishes.length}</b><span>요리 수</span></div>
    <div class="fact"><b>${R.groupsCovered}/5</b><span>식품군</span></div>
    <div class="fact"><b>${R.servings ? R.servings.toLocaleString() : '-'}</b><span>급식 인원</span>
</div>
  </div>`;
}
```

✍️ 3) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 식품군 구성 색깔 띠 */
function renderGroups(A, B) {
  const groups = [...FOOD_GROUPS, GROUP_ETC];

  $('#groupLegend').innerHTML = groups
    .map(g => `<span><i style="background:${g.color}"></i>${esc(g.label)}</span>`).join('');

  const stack = (R, cls) => {
    const total = R.dishes.length || 1;
    const segs = groups.filter(g => R.groupCount[g.key] > 0).map(g => {
      const n = R.groupCount[g.key];
      const pct = (n / total) * 100;
      return `<div style="background:${g.color};width:${pct}%" title="${esc(g.label)} ${n}가
지">${pct > 12 ? n : ''}</div>`;
    }).join('');
    return `<div class="group-row">
      <div class="bar-label"><span class="chip chip-${cls}">${cls.toUpperCase()}</span>
        ${esc(R.school.SCHUL_NM)} — 요리 ${R.dishes.length}가지, 식품군 ${R.groupsCovered}/5</div>
      <div class="stack">${segs}</div>
    </div>`;
  };
  $('#groupStacks').innerHTML = stack(A, 'a') + stack(B, 'b');
}
```

✍️ 4) `renderAll` 함수 안의 `★★★` 아래에 한 줄 추가

```
  renderGroups(A, B);
```

💾 **Ctrl+S** → 🔄 **F5** → **비교하기**

👀 **확인!**

1. 숫자 상자가 **4개**로 늘고 `5/5` **식품군** 이 생겼습니다.
2. 🧺 **식품군 구성** 칸에 **알록달록한 색깔 띠**가 두 줄 나타납니다!

☐ **성공!** 색깔 띠가 나왔어요.

🔍 미션! 컴퓨터의 실수를 찾아라

컴퓨터는 요리의 뜻을 모릅니다. 글자만 봅니다. 그래서 **반드시 실수**를 해요.

🎬 **예시 데이터로 먼저 보기** 버튼은 아직 안 만들었으니, 아래 표를 먼저 보세요.

요리 이름	컴퓨터의 분류	맞나요?	왜 그렇게 됐을까?
리코타과일샐러드	**우유·유제품**	❓	
단호박크림스프	**채소류**	❓	
감자치즈롤까스	**고기·생선·달걀·콩**	❓	

우리 학교 급식에서도 찾아서 3개 이상 적어 보세요!

요리 이름	컴퓨터의 분류	내 생각	어떤 낱말 때문일까?

✍️ **직접 고쳐 보기** `FOOD_GROUPS` 의 `words` 목록에 낱말을 넣거나 빼고 → **저장 → F5** → 결과가 바뀝니다!

> 🤖 **여기서 배우는 것 — 아주 중요해요!**
>
> 이렇게 **사람이 낱말을 하나하나 정해 주는 방식**을 **규칙 기반**이라고 합니다. 예외가 너무 많아서 힘들죠? 아무리 낱말을 늘려도 새로운 요리가 계속 나옵니다.
>
> 그래서 요즘은 **인공지능(AI)** 에게 **수많은 예를 보여 주고 스스로 규칙을 찾게** 합니다. 여러분은 지금 **"AI가 왜 필요한가"** 를 몸으로 배운 거예요! 👏

🎊 4장 완성!

> 🆘 막혔다면 → `steps/4단계_app.js` 로 갈아타고 마지막 장으로!

🏅 5장. 점수 만들기

마지막 장입니다! 그리고 **가장 중요한 것**을 배웁니다.

🧐 먼저 생각해 보기 (코딩 전에!)

"좋은 급식"이란 무엇일까요? 모둠에서 3개 이상 적어 보세요.

1. _____
2. _____
3. _____

우리 앱은 이 5가지로 정했습니다. **여러분 생각과 같나요?**

항목	어떻게 점수를 낼까
① 열량 적정성	권장 열량에 **가까울수록** 100점 (많아도 감점!)
② 단백질 충분도	권장 단백질을 채우면 100점
③ 식단 다양성	식품군 수(5개 중 몇 개) + 요리 수
④ 미량영양소	칼슘·철분·비타민A·C 의 달성률 평균
⑤ 가공·튀김 절제	가공/튀김 요리가 **적을수록** 100점

⭐ 미션 18. 급식 점수 매기기

✍️ 1) ②번 자료 칸 맨 아래에 붙여넣기

```
/* 점수 항목 5개 */
const AXES = [
  { key: 'energy',  label: '열량 적정성',   short: '열량',       desc: '권장 열량에 가까울수록 높음' },
  { key: 'protein', label: '단백질 충분도', short: '단백질',     desc: '권장 단백질을 채웠는지' },
  { key: 'variety', label: '식단 다양성',   short: '다양성',     desc: '식품군 수 + 요리 수' },
  { key: 'micro',   label: '미량영양소',    short: '미량영양소', desc: '칼슘·철분·비타민A·C 달성률' },
  { key: 'fresh',   label: '가공·튀김 절제', short: '가공·튀김',  desc: '가공/튀김 요리가 적을수록 높
음' },
];

/* 항목 중요도(가중치) — 미션 20에서 막대로 바꿀 수 있게 만듭니다 */
let WEIGHTS = { energy: 25, protein: 20, variety: 20, micro: 20, fresh: 15 };
```

✍️ 2) ⑤번 계산하기 칸의 `analyze` 함수 위쪽에 붙여넣기

```
/* ① 열량 적정성 — 권장 열량에 가까울수록 100점 (많아도 감점!) */
function scoreEnergy(R) {
  if (R.kcal == null || !R.std.에너지) return null;
  const ratio = R.kcal / R.std.에너지;                 // 1.0 이면 딱 맞음
  return clamp(100 - Math.abs(ratio - 1) * 250, 0, 100);
}

/* ② 단백질 충분도 — 권장 단백질을 채우면 100점 */
function scoreProtein(R) {
  const p = R.nutri['단백질'];
  if (!p || !R.std.단백질) return null;
  return clamp((p.value / R.std.단백질) * 100, 0, 100);
}

/* ③ 식단 다양성 — 식품군 수(70%) + 요리 수(30%) */
function scoreVariety(R) {
  if (!R.dishes.length) return null;
  return clamp((R.groupsCovered / 5) * 70 + (Math.min(R.dishes.length, 7) / 7) * 30, 0, 100);
}

/* ④ 미량영양소 — 칼슘·철분·비타민A·비타민C 달성률의 평균 */
function scoreMicro(R) {
  const rates = ['칼슘', '철분', '비타민A', '비타민C']
    .filter(k => R.nutri[k] && R.std[k])
    .map(k => clamp((R.nutri[k].value / R.std[k]) * 100, 0, 100));
  return rates.length ? rates.reduce((a, b) => a + b, 0) / rates.length : null;
}

/* ⑤ 가공·튀김 절제 — 가공/튀김 비율이 낮을수록 100점 */
function scoreFresh(R) {
  if (!R.dishes.length) return null;
  return clamp(100 - (R.processedCnt / R.dishes.length) * 200, 0, 100);
}

/* 점수 5개를 중요도로 합쳐 총점 내기 (자료 없는 항목은 빼고 계산) */
function totalScore(scores) {
  let sum = 0, w = 0;
  for (const ax of AXES) {
    const v = scores[ax.key];
    if (v == null) continue;
    sum += v * WEIGHTS[ax.key];
    w += WEIGHTS[ax.key];
  }
  return w ? sum / w : null;
}
```

✍️ 3) `analyze` 함수 안의 `★★★` 아래(미션 17에서 넣은 것 다음)에 붙여넣기

```
  /* 점수 5가지 */
  R.scores = {
    energy:  scoreEnergy(R),
    protein: scoreProtein(R),
    variety: scoreVariety(R),
    micro:   scoreMicro(R),
    fresh:   scoreFresh(R),
  };
```

✍️ 4) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 총점 판(원형 게이지) 그리기 */
function renderScoreboard(A, B) {
  const card = (R, cls, colorVar) => {
    const t = totalScore(R.scores);
    return `<div class="score-card">
      <h3><span class="chip chip-${cls}">학교 ${cls.toUpperCase()}</span> ${esc(R.school.SCHUL_NM)}
        <small>${esc(R.school.SCHUL_KND_SC_NM || '')} · ${esc(STANDARD[R.levelKey].label)} 기준 적용
</small></h3>
      <div class="gauge" style="--p:${t == null ? 0 : t.toFixed(1)}; --c:var(${colorVar})">
        <b>${t == null ? '–' : Math.round(t)}<i>점</i></b>
      </div>
    </div>`;
  };
  $('#scoreboard').innerHTML = card(A, 'a', '--a') + card(B, 'b', '--b');

  const tA = totalScore(A.scores), tB = totalScore(B.scores);
  const diff = (tA != null && tB != null) ? Math.abs(tA - tB) : null;
  let msg = `${fromYmd(A.date)} ${esc(A.mealName)} 기준입니다.`;
  if (diff != null) {
    msg += diff < 3
      ? ' 두 학교의 총점 차이가 3점 미만이라 <b>거의 비슷</b>합니다.'
      : ` 총점 차이는 <b>${round1(diff)}점</b>입니다.`;
  }
  $('#scoreSub').innerHTML = msg;
}
```

✍️ 5) `renderAll` 함수 안의 `★★★` 아래에 한 줄 추가

```
  renderScoreboard(A, B);
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 **확인!** 맨 위에 **원형 점수판 2개**가 나타납니다! 색깔 고리가 점수만큼 채워져 있어요.

```
  학교 A                학교 B
경기초등학교          창원고등학교
  ╭──╮                  ╭──╮
  │87점│                 │67점│
  ╰──╯                  ╰──╯
```

▢ **성공!** 점수가 나왔어요! 🎉

> 🔰 `100 - Math.abs(ratio - 1) * 250` **이게 뭐죠?**
>
> `Math.abs` 는 **절댓값**(부호 없애기)입니다. `-0.2` → `0.2`
>
권장 대비	ratio	차이	점수
딱 맞음	1.0	0	**100점**
20% 많음	1.2	0.2	50점
20% 적음	0.8	0.2	50점
40% 많음	1.4	0.4	**0점**
>
> **너무 적어도, 너무 많아도 감점**입니다. 왜 그렇게 만들었을까요? 이야기해 보세요!
>
> 🔥 `250` 이라는 숫자는 **우리가 그냥 정한 것**입니다. `150` 으로 바꾸면 점수가 후해져요. 한번 바꿔 보세요! **이렇게 사람이 정하는 숫자를 어떻게 정할지가 데이터 분석의 진짜 어려움입니다.**
>
> `return null` → "계산할 자료가 없어요"라는 뜻. 총점 계산에서 **빠집니다**. `reduce((a, b) => a + b, 0)` → 배열의 값을 **전부 더하기**. `[10,20,30]` → `60`

✅ 미션 19. 거미줄 차트 그리기

✍️ 1) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 항목별 점수 막대 */
function renderScoreBars(A, B) {
  $('#scoreBars').innerHTML = AXES
    .map(ax => barRow(ax.label, ax.desc, A.scores[ax.key], B.scores[ax.key], 100, '점'))
    .join('');
}

/* 레이더(거미줄) 차트 그리기 */
function renderRadar(A, B) {
  const size = 340, c = size / 2, R = 108, n = AXES.length;

  /* i번째 축의, 중심에서 r만큼 떨어진 점의 좌표 */
  const pt = (i, r) => {
    const ang = -Math.PI / 2 + (i * 2 * Math.PI) / n;   // 12시 방향부터 시계방향
    return [c + Math.cos(ang) * r, c + Math.sin(ang) * r];
  };
  const poly = vals => vals
    .map((v, i) => pt(i, (clamp(v ?? 0, 0, 100) / 100) * R).map(x => x.toFixed(1)).join(','))
    .join(' ');

  /* 배경 거미줄 (25/50/75/100%) */
  const rings = [25, 50, 75, 100].map(p => {
    const d = AXES.map((_, i) => pt(i, (p / 100) * R).map(x => x.toFixed(1)).join(',')).join(' ');
    return `<polygon points="${d}" fill="none" stroke="var(--line)" stroke-width="1"/>`;
  }).join('');

  /* 중심에서 뻗어 나가는 선 + 축 이름 */
  const spokes = AXES.map((ax, i) => {
    const [x, y] = pt(i, R);
    const [lx, ly] = pt(i, R + 24);
    const anchor = lx > c + 6 ? 'start' : lx < c - 6 ? 'end' : 'middle';
    return `<line x1="${c}" y1="${c}" x2="${x.toFixed(1)}" y2="${y.toFixed(1)}"
              stroke="var(--line)" stroke-width="1"/>
      <text x="${lx.toFixed(1)}" y="${(ly + 4).toFixed(1)}" text-anchor="${anchor}"
            font-size="11" font-weight="700" fill="var(--muted)">${esc(ax.short)}</text>`;
  }).join('');

  const vA = AXES.map(ax => A.scores[ax.key]);
  const vB = AXES.map(ax => B.scores[ax.key]);

  $('#radar').innerHTML = `
    <svg viewBox="-30 -18 ${size + 60} ${size + 36}" role="img"
         aria-label="다섯 항목 점수를 겹쳐 그린 레이더 차트">
      ${rings}${spokes}
      <polygon points="${poly(vB)}" fill="var(--b)" fill-opacity="0.22"
               stroke="var(--b)" stroke-width="2"/>
      <polygon points="${poly(vA)}" fill="var(--a)" fill-opacity="0.22"
               stroke="var(--a)" stroke-width="2"/>
      ${vA.map((v, i) => { const [x, y] = pt(i, (clamp(v ?? 0, 0, 100) / 100) * R);
        return `<circle cx="${x.toFixed(1)}" cy="${y.toFixed(1)}" r="3.5" fill="var(--a)"/>`;
}).join('')}
```

```
      ${vB.map((v, i) => { const [x, y] = pt(i, (clamp(v ?? 0, 0, 100) / 100) * R);
        return `<circle cx="${x.toFixed(1)}" cy="${y.toFixed(1)}" r="3.5" fill="var(--b)"/>`;
}).join('')}
    </svg>`;
}
```

✍️ 2) `renderAll` 함수 안의 `★★★` 아래에 두 줄 추가

```
  renderScoreBars(A, B);
  renderRadar(A, B);
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 **확인!** **항목별 점수 비교** 칸에 **오각형 거미줄 그림**이 나타나고, 파란색·주황색 오각형이 **겹쳐서** 그려집니다! 오른쪽에는 항목별 막대도 5줄 나와요.

▢ **성공!** 거미줄 차트를 그렸어요! 🎉

> **🔰 삼각함수를 진짜로 써먹었어요! 📐**
>
> ```
> const ang = -Math.PI / 2 + (i * 2 * Math.PI) / n;
> return [c + Math.cos(ang) * r, c + Math.sin(ang) * r];
> ```
>
> **원 위의 점을 찍는 공식**입니다.
>
> ```
> x = 중심 + cos(각도) × 반지름
> y = 중심 + sin(각도) × 반지름
> ```
>
> - `Math.PI` = 원주율 π (180°)
> - `-Math.PI/2` = −90° → **12시 방향**부터 시작하려고 뺐어요
> - `2π ÷ 5` = 축 5개를 **똑같은 간격**(72°씩)으로 벌리기
>
> 🎓 **"삼각함수 배워서 어디에 써요?"** 의 답이 바로 이것입니다!
>
> `<svg>` 는 **그림을 그리는 HTML**이에요. `<polygon>` = 여러 점을 이은 도형 / `<line>` = 선 / `<circle>` = 동그라미 / `<text>` = 글씨

⭐ 미션 20. 내 기준으로 순위 뒤집기 — 이 수업의 결론!

✍️ 1) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 중요도 조절 막대(슬라이더) 만들기 */
function renderWeights(onChange) {
  $('#weights').innerHTML = AXES.map(ax => `
    <div class="weight-row">
      <label for="w-${ax.key}">${esc(ax.label)}</label>
      <input type="range" id="w-${ax.key}" min="0" max="40" step="5" value="${WEIGHTS[ax.key]}"
data-wkey="${ax.key}">
      <output id="o-${ax.key}">${WEIGHTS[ax.key]}%</output>
    </div>`).join('');

  $$('#weights input[type="range"]').forEach(inp => {
    inp.addEventListener('input', () => {
      WEIGHTS[inp.dataset.wkey] = Number(inp.value);       // 중요도 바꾸고
      $('#o-' + inp.dataset.wkey).textContent = inp.value + '%';
      onChange();                                          // 총점 다시 계산!
    });
  });
}

/* 중요도만 바뀌었을 때 총점만 다시 그리기 */
function rescore() {
  if (!CURRENT) return;
  renderScoreboard(CURRENT.A, CURRENT.B);
}
```

✍️ 2) ⑧번 시작 칸의 `★★★` 아래에 추가

```
  renderWeights(rescore);
```

💾 **Ctrl+S** → 🔄 **F5** → **비교하기**

👀 **확인!** ⚖️ **항목 중요도 바꿔 보기** 칸에 **막대 5개**가 나타납니다. 막대를 **좌우로 끌어 보세요** → 위쪽 **원형 점수판의 숫자가 즉시 바뀝니다!**

☐ **성공!** 슬라이더로 점수가 바뀌어요! 🎉

🎯 마지막 미션 — 순위를 뒤집어 보세요!

1. 지금 총점을 적으세요. → **A: _____점 / B: _____점**
2. **슬라이더를 움직여서 순위가 뒤바뀌게** 만들어 보세요.
3. 어떤 항목의 중요도를 올렸나요? _____

4. 그렇게 정한 **이유**는 무엇인가요? _____

> 💭 **이게 이 수업의 진짜 결론입니다**
>
> **가중치를 바꾸면 1등이 바뀝니다.**
>
> 세상의 모든 순위 — 학교 순위, 맛집 순위, 앱 추천, 심지어 성적 계산까지 — 는 **누군가가 "무엇이 중요한가"를 정한 결과**입니다.
>
> 그래서 우리는 순위를 볼 때 항상 이렇게 물어야 합니다.
>
> 👉 **"이 점수는 누가, 무엇을 중요하게 여겨서 만들었나요?"**

✅ 미션 21. 자동 분석 글 만들기

✍️ 1) ⑥번 화면 그리기 칸 맨 아래에 붙여넣기

```
/* 자동 분석 코멘트 + 토론 질문 */
function renderComments(A, B) {
  const out = [];
  const nameA = A.school.SCHUL_NM, nameB = B.school.SCHUL_NM;

  /* 열량 */
  if (A.kcal != null && B.kcal != null) {
    const d = A.kcal - B.kcal;
    out.push(['info', `열량은 <b>${esc(nameA)} ${round1(A.kcal)}kcal</b>,
      <b>${esc(nameB)} ${round1(B.kcal)}kcal</b>로 차이가 ${round1(Math.abs(d))}kcal입니다.`]);

    for (const R of [A, B]) {
      const r = R.kcal / R.std.에너지;
      if (r > 1.25) out.push(['down', `${esc(R.school.SCHUL_NM)}은(는) ${esc(R.std.label)} 권장 열량
(${R.std.에너지}kcal)의
        <b>${Math.round(r * 100)}%</b>로 다소 높습니다. 후식이나 튀김 요리가 있었는지 확인해 보세요.`]);
      else if (r < 0.8) out.push(['down', `${esc(R.school.SCHUL_NM)}은(는) 권장 열량의
<b>${Math.round(r * 100)}%</b>로
        다소 낮습니다. 일품요리(한 그릇 음식)만 나온 날일 수 있습니다.`]);
      else out.push(['up', `${esc(R.school.SCHUL_NM)}의 열량은 권장의 <b>${Math.round(r * 100)}%</b>
로 적정 범위(80~120%)입니다.`]);
    }
  }

  /* 단백질 */
  for (const R of [A, B]) {
    const p = R.nutri['단백질']?.value;
    if (p == null) continue;
    const r = p / R.std.단백질;
    out.push([r >= 1 ? 'up' : 'down',
      `${esc(R.school.SCHUL_NM)}의 단백질은 <b>${round1(p)}g</b>으로 권장(${R.std.단백질}g)의
${Math.round(r * 100)}%입니다.`]);
  }

  /* 다양성 */
  const varyMore = A.groupsCovered === B.groupsCovered ? null : (A.groupsCovered > B.groupsCovered ?
A : B);
  if (varyMore) {
    out.push(['info', `식품군은 <b>${esc(varyMore.school.SCHUL_NM)}</b>이(가) 더 골고루 들어갔습니다
      (${A.groupsCovered}/5 vs ${B.groupsCovered}/5).`]);
  } else {
    out.push(['info', `두 학교 모두 식품군 <b>${A.groupsCovered}/5</b>로 같습니다.`]);
  }

  /* 빠진 식품군 알려 주기 */
  for (const R of [A, B]) {
    const missing = FOOD_GROUPS.filter(g => !R.groupCount[g.key]).map(g => g.label);
    if (missing.length) out.push(['down',
      `${esc(R.school.SCHUL_NM)}의 식단에서는 <b>${esc(missing.join(', '))}</b>(으)로 분류된 요리를 찾
지 못했습니다.
       정말 없었을까요, 아니면 컴퓨터가 못 알아본 걸까요?`]);
```

```
  }

  /* 가공식품 */
  for (const R of [A, B]) {
    if (!R.dishes.length) continue;
    const pct = Math.round((R.processedCnt / R.dishes.length) * 100);
    out.push([pct >= 34 ? 'down' : 'up',
      `${esc(R.school.SCHUL_NM)}은(는) ${R.dishes.length}가지 중 <b>${R.processedCnt}가지(${pct}%)
</b>가
       가공·튀김 낱말에 걸렸습니다.`]);
  }

  /* 원산지 */
  const originDiff = [...new Set([...Object.keys(A.origin), ...Object.keys(B.origin)])]
    .filter(k => (A.origin[k] || '') !== (B.origin[k] || ''));
  if (originDiff.length) out.push(['info',
    `원산지 표기가 다른 품목이 <b>${originDiff.length}개</b> 있습니다
     (예: ${esc(originDiff.slice(0, 3).join(', '))}).`]);

  $('#comments').innerHTML = out
    .map(([k, html]) => `<li class="${k}">${html}</li>`).join('');

  /* 토론 질문 */
  $('#questions').innerHTML = [
    `두 학교의 열량 차이가 생긴 가장 큰 이유는 어떤 요리 때문일까요?`,
    `‘좋은 급식’을 점수로 나타낼 때, 여러분은 어떤 항목에 가장 큰 중요도를 주고 싶나요? 위 슬라이더를 움직
여 보고 그 이유를 적어 보세요.`,
    `컴퓨터가 식품군을 <b>잘못 분류한 요리</b>를 찾아보세요. 왜 틀렸을까요? app.js의 FOOD_GROUPS에 어떤
낱말을 넣으면 고칠 수 있을까요?`,
    `알레르기 번호가 많은 식단은 나쁜 급식일까요? 왜 그렇게 생각하나요?`,
    `${esc(A.school.SCHUL_KND_SC_NM || 'A')}와 ${esc(B.school.SCHUL_KND_SC_NM || 'B')}의 권장 기준이
다른 이유는 무엇일까요?`,
    `같은 학교의 다른 날짜를 비교하면 결과가 어떻게 달라질까요? 한 주 동안 조사해 보세요.`,
  ].map(q => `<li>${q}</li>`).join('');
}
```

✍️ 2) `renderAll` 함수 안의 `★★★` 아래에 한 줄 추가

```
  renderComments(A, B);
```

💾 Ctrl+S → 🔄 F5 → 비교하기

👀 **확인!** 맨 아래 💬 **자동 분석 결과** 칸에 문장이 여러 줄 나타납니다. 왼쪽에 **초록(좋음)·빨강(주의)·파랑(정보)** 색 줄이 붙어 있어요! 그 아래에는 🤔 **함께 생각해 볼 질문** 6개가 나옵니다.

▢ **성공!** 컴퓨터가 분석 글을 써 줬어요! 🎉

> 🔰 **컴퓨터가 글을 쓴다고요?**
>
> 사실 **AI가 아니라 우리가 미리 정해 둔 규칙**입니다.
>
> ```
> if (r > 1.25)       → "다소 높습니다"
> else if (r < 0.8)   → "다소 낮습니다"
> else                → "적정 범위입니다"
> ```
>
> **조건에 따라 미리 준비한 문장을 골라 주는 것**뿐이에요. 하지만 이것만으로도 꽤 똑똑해 보이죠? 😄

✅ 미션 22. 마무리 기능 5가지

거의 다 왔습니다! 편리한 기능들을 **하나씩** 붙여 봅시다.

🅰 인쇄 버튼 (딱 1줄!)

⑧번 시작 칸의 `★★★` 아래에 추가

```
$('#printBtn').addEventListener('click', () => window.print());
```

👀 **확인!** 아래쪽 🖨️ **인쇄 / PDF로 저장** 버튼을 누르면 인쇄 창이 뜹니다.

▢ 성공!

🅱 밤/낮 모드 바꾸기

⑦번 버튼 연결 칸에 붙여넣기

```
/* 밝은 화면 / 어두운 화면 바꾸기 */
function setupTheme() {
  const btn = $('#themeBtn');
  const apply = t => {
    if (t) document.documentElement.dataset.theme = t;
    else delete document.documentElement.dataset.theme;
    btn.textContent = t === 'dark' ? '🌞' : '🌙';
  };
  apply(localStorage.getItem('owlTheme') || '');
  btn.addEventListener('click', () => {
    const now = document.documentElement.dataset.theme === 'dark' ? 'light' : 'dark';
    localStorage.setItem('owlTheme', now);
    apply(now);
    if (CURRENT) renderRadar(CURRENT.A, CURRENT.B);   // 차트 색 다시 맞추기
  });
}
```

⑧번 시작 칸의 `★★★` 아래에 추가

```
setupTheme();
```

👀 **확인!** 오른쪽 위 🌙 버튼을 누르면 화면이 **까맣게** 바뀝니다! 다시 누르면 밝아져요.

▢ 성공!

🅒 CSV로 저장하기 (엑셀에서 열려요)

⑦**번 버튼 연결 칸**에 붙여넣기

```
/* 결과를 CSV 파일로 저장 */
function downloadCsv() {
  if (!CURRENT) return;
  const { A, B } = CURRENT;
  const q = v => `"${String(v ?? '').replace(/"/g, '""')}"`;
  const lines = [];

  lines.push(['구분', A.school.SCHUL_NM, B.school.SCHUL_NM].map(q).join(','));
  lines.push(['날짜', fromYmd(A.date), fromYmd(B.date)].map(q).join(','));
  lines.push(['끼니', A.mealName, B.mealName].map(q).join(','));
  lines.push(['학교급', A.school.SCHUL_KND_SC_NM, B.school.SCHUL_KND_SC_NM].map(q).join(','));
  lines.push(['적용 기준', A.std.label, B.std.label].map(q).join(','));
  lines.push(['식단', A.dishes.map(d => d.name).join(' / '), B.dishes.map(d => d.name).join(' /
')].map(q).join(','));
  lines.push(['요리 수', A.dishes.length, B.dishes.length].map(q).join(','));
  lines.push(['식품군 수(5)', A.groupsCovered, B.groupsCovered].map(q).join(','));
  lines.push(['가공·튀김 요리 수', A.processedCnt, B.processedCnt].map(q).join(','));
  lines.push(['급식 인원', A.servings, B.servings].map(q).join(','));

  lines.push('');
  lines.push(['[영양소]', '', ''].map(q).join(','));
  for (const n of NUTRI_ORDER) {
    const a = A.nutri[n], b = B.nutri[n];
    if (!a && !b) continue;
    lines.push([`${n}(${a?.unit || b?.unit || ''})`, a?.value ?? '', b?.value ??
''].map(q).join(','));
  }

  lines.push('');
  lines.push(['[항목 점수]', '', '중요도(%)'].map(q).join(','));
  for (const ax of AXES) {
    lines.push([ax.label,
      A.scores[ax.key] == null ? '' : round1(A.scores[ax.key]),
      B.scores[ax.key] == null ? '' : round1(B.scores[ax.key]),
      WEIGHTS[ax.key]].map(q).join(','));
  }
  const tA = totalScore(A.scores), tB = totalScore(B.scores);
  lines.push(['총점', tA == null ? '' : round1(tA), tB == null ? '' : round1(tB)].map(q).join(','));

  /* 맨 앞에 특별한 표시를 붙이면 엑셀에서 한글이 안 깨집니다 */
  const blob = new Blob(['' + lines.join('\r\n')], { type: 'text/csv;charset=utf-8' });
  const a = document.createElement('a');
  a.href = URL.createObjectURL(blob);
  a.download = `급식비교_${A.school.SCHUL_NM}_${B.school.SCHUL_NM}_${A.date}.csv`;
  a.click();
  URL.revokeObjectURL(a.href);
}
```

⑧번 시작 칸의 `★★★` 아래에 추가

```
  $('#csvBtn').addEventListener('click', downloadCsv);
```

👀 **확인!** 💾 **결과를 CSV로 저장** 버튼 → 다운로드 폴더에 파일이 생깁니다. 엑셀로 열어 보세요!

▢ 성공!

🅳 급식 있는 날 자동으로 찾기

⑦번 버튼 연결 칸에 붙여넣기

```
/* 고른 날짜에서 거꾸로 세면서, 두 학교 모두 급식이 있는 날 찾기 */
async function findRecent() {
  const A = PICKED.A, B = PICKED.B;
  if (!A || !B) { setStatus('먼저 학교 A와 B를 골라 주세요.', 'warn'); return; }

  const type = $('#mealType').value;
  const start = $('#date').value ? new Date($('#date').value) : new Date();

  $('#recentBtn').disabled = true;
  let checked = 0;
  try {
    for (let back = 0; back <= 30 && checked < 15; back++) {
      const d = new Date(start);
      d.setDate(d.getDate() - back);
      if (d.getDay() === 0 || d.getDay() === 6) continue;      // 토·일은 건너뛰기
      checked++;

      const p = n => String(n).padStart(2, '0');
      const ymd = `${d.getFullYear()}${p(d.getMonth() + 1)}${p(d.getDate())}`;
      setStatus(`${fromYmd(ymd)} 확인 중… (${checked}번째)`, 'loading');

      const [ra, rb] = await Promise.all([fetchMeal(A, ymd), fetchMeal(B, ymd)]);
      const ok = ra.some(r => r.MMEAL_SC_CODE === type) && rb.some(r => r.MMEAL_SC_CODE === type);
      if (ok) {
        $('#date').value = fromYmd(ymd);
        setStatus(`두 학교 모두 급식이 있는 <b>${fromYmd(ymd)}</b>을(를) 찾았습니다. 이제 비교합니
다.`);
        await compare();
        return;
      }
    }
    setStatus('최근 15번의 평일에서는 두 학교가 함께 급식한 날을 찾지 못했습니다. 날짜를 학기 중으로 바꿔
보세요.', 'warn');
  } catch (e) {
    showError(e);
  } finally {
    $('#recentBtn').disabled = false;
  }
}
```

⑧번 시작 칸의 `★★★` 아래에 추가

```
  $('#recentBtn').addEventListener('click', findRecent);
```

👀 **확인!** 날짜를 **토요일**로 바꾸고 🔎 **급식 있는 날 찾기** 를 눌러 보세요. 날짜가 저절로 **금요일**로 바뀌면서 비교까지 됩니다!

▢ 성공!

🔰 `d.getDay()` 는 요일을 숫자로 줍니다. **0=일요일, 6=토요일** `continue` 는 **"이번 건 건너뛰고 다음으로"** 라는 뜻이에요.

🅴 예시 데이터 버튼 (인터넷이 안 될 때 대비)

이 코드는 조금 기니까, `steps/5단계_완성_app.js` 파일을 열어서 `const DEMO = {` 부터 `function showDemo() { … }` 끝까지 복사해 **⑦번 버튼 연결 칸**에 붙여넣으세요.

⑧번 시작 칸의 `★★★` 아래에 추가

```
  $('#demoBtn').addEventListener('click', showDemo);
```

👀 **확인!** 🎬 **예시 데이터로 먼저 보기** 버튼을 누르면 인터넷 없이도 결과가 나옵니다!

- [ ] 성공!

🏆 완성! 정말 대단합니다!

마지막으로 `init` 함수를 확인해 보세요

여러분의 **⑧번 시작 칸**은 이런 모습일 거예요. (순서는 조금 달라도 괜찮습니다!)

```
function init() {
  $('#date').value = todayISO();

  setupTheme();
  setupKeyBox();
  setupPicker('A');
  setupPicker('B');
  renderWeights(rescore);

  $('#compareBtn').addEventListener('click', compare);
  $('#recentBtn').addEventListener('click', findRecent);
  $('#demoBtn').addEventListener('click', showDemo);
  $('#csvBtn').addEventListener('click', downloadCsv);
  $('#printBtn').addEventListener('click', () => window.print());

  setStatus('학교 두 곳을 검색해서 고른 뒤 <b>📊 두 학교 비교하기</b>를 누르세요. ' +
            '처음이라면 <b>🎬 예시 데이터로 먼저 보기</b>부터 눌러 보세요.');
}

init();
```

- [ ] **미션 22 성공! 앱 완성!!** 🎉🎉🎉

🎓 마무리

내가 만든 것

📡	인터넷에서 **전국 학교 자료**를 가져오는 프로그램
🎀	긴 글자를 쪼개 **쓸 수 있는 자료**로 바꾸는 기능
📊	라이브러리 없이 **직접 그린** 막대그래프와 거미줄 차트
🏅	내가 기준을 정하는 **점수 계산기**
💬	자동으로 **분석 글**을 써 주는 기능

이걸 다 만들었습니다. 정말 대단해요! 👏

⚠️ 꼭 기억할 것

> 이 점수는 **우리가 수업에서 직접 만든 기준**입니다. 공식 평가가 아니에요. 학교마다 학생 수, 급식 예산, 지역 사정, 학교급이 다릅니다. **점수가 낮다고 그 학교 급식이 나쁜 것이 절대 아닙니다.**
>
> 데이터는 **답을 주는 것이 아니라, 더 좋은 질문을 만들어 주는 도구**입니다. 🦉

🚀 더 해보기 (도전 과제)

난이도	과제
⭐	`FOOD_GROUPS` 에 낱말을 20개 이상 더 넣어 오분류 줄이기
⭐	`style.css` 의 `--a`, `--b` 색을 바꿔 나만의 색으로
⭐	점수 계산의 `250` 을 `150` 으로 바꿔 보고 결과 비교하기
⭐⭐	새 점수 항목 추가하기 (예: 국내산 비율 점수)
⭐⭐	같은 학교의 **한 주 열량 변화**를 표로 만들기
⭐⭐⭐	학교를 **3곳 이상** 비교할 수 있게 고치기
⭐⭐⭐	내 알레르기 번호를 고르면 그 요리를 **빨갛게** 표시하기

📖 배운 낱말 정리

낱말	뜻	어디서 썼나
API	프로그램끼리 자료를 주고받는 약속	NEIS에서 급식 받아오기
JSON	자료를 적는 정해진 모양	NEIS가 준 답
fetch	인터넷에서 자료 받아오기	`callNeis`
async / await	오래 걸리는 일을 기다리기	급식 불러오기
파싱(parsing)	긴 글자를 쓸 수 있는 자료로 쪼개기	`parseDishes`
정규식	"이런 모양의 글자를 찾아줘"	알레르기 번호 떼기
정규화	서로 다른 크기를 비교하게 맞추기	막대 길이(%)
가중치	무엇을 더 중요하게 볼지 정하는 무게	총점 계산
규칙 기반 분류	사람이 정한 규칙으로 나누기	식품군 분류
데이터 시각화	숫자를 그림으로 보여 주기	막대·거미줄 차트

✍️ 오늘의 소감

가장 신기했던 순간은?

가장 어려웠던 부분과, 어떻게 넘겼는지?

이 앱에 더 넣고 싶은 기능은?